

分类号：TP301.6

密级：公开

兰州大学

研究生学位论文

论文题目（中文）	挖掘任意形状簇的聚类算法研究
论文题目（外文）	The Research of Clustering Algorithms for Mining Arbitrary Shaped Clusters
研 究 生 姓 名	王博
学 科 、 专 业	软件工程
研 究 方 向	数据仓库与数据挖掘
学 位 级 别	硕 士
导师姓名、职称	陈晓云 教授
论文工作起止年月	2014 年 9 月至 2016 年 5 月
论 文 提 交 日 期	2016 年 4 月
论 文 答 辩 日 期	2016 年 5 月
学 位 授 予 日 期	

校址：甘肃省兰州市

挖掘任意形状簇的聚类算法研究

摘要

数据挖掘是一个能从大规模数据中发现知识的强大技术。聚类分析作为数据挖掘中的一个基础工具,已经被广泛应用于模式识别、图像处理、空间数据分析、文本分类和信息检索、市场分析等众多领域。随着计算机的普及和互联网的不断发展,数据量越来越大,数据的空间分布中大多包含多种不规则形状的簇,比如地理信息数据、医学图像数据、农业科学数据等,这对聚类分析提出了挑战。传统的聚类算法通常不能很好地挖掘任意形状的簇,近年来,挖掘任意形状簇的研究成为聚类分析领域的一个研究热点。为了更有效地对包含有任意形状簇的数据进行聚类,本文对现有的聚类算法进行分析和研究,提出了两个能挖掘任意形状簇的聚类算法 CMSPC、CFDPm。

本文提出的 CMSPC 算法是为了提高在包含任意形状簇的数据集上的聚类质量。CMSPC 算法基于点与簇内多点的相似性,对于距离在截断距离之内的两个对象,考虑其中一个对象关于另外一个对象所在簇的归属度,对满足一定归属度的临时簇进行合并。CMSPC 算法基于点与簇内多点相似的特点使得聚类结果与簇形状无关,同时能够提取出异常点。本文提出的另外一个聚类算法 CFDPm 是新型聚类算法 CFDP 算法的一种改进算法。在多峰簇数据集中,CFDP 算法会因簇中心点的选取不准确而导致聚类质量的下降。本文通过综合考虑聚类结果中簇之间的距离、簇内距离、合并两簇后对于整体内部评价指标 DBI 的影响来有条件地对簇进行合并,从而改善因选择簇中心点失效而带来的聚类质量下降的问题。

为了验证本文提出的两种聚类算法的有效性,我们在多个基准数据集上进行了聚类实验。实验结果表明两点:一、CMSPC 算法可以对包含任意形状簇的数据进行较好的聚类,同时识别出异常点,具有较高的聚类质量。二、CFDPm 算法能够克服 CFDP 算法因簇中心选取不准确而导致的聚类质量下降的问题。

关键词: 任意形状簇, 聚类, 空间数据, 相似性度量

The Research of Clustering Algorithms for Mining Arbitrary Shaped Clusters

Abstract

Data mining is a powerful technology which can discover knowledge from massive data. Cluster analysis, as a basic tool for data mining, has been applied in many areas, such as pattern recognition, image processing, spatial data analysis, text categorization and information retrieval, market analysis and so on. With the wide access of computers and development of the Internet, the amount of data is more and more increasing, and the spatial distribution of data includes many kinds of clusters with irregular shapes, such as geographic information data, medical image data, agricultural science data and so on, which brings great challenges for cluster analysis. Generally, traditional clustering algorithm cannot detect arbitrary shaped clusters well. Recently, the research on mining arbitrary shaped clusters become a focus in the area of cluster analysis. In order to cluster data containing arbitrary shaped clusters more effectively, We analyzed and studied available clustering algorithms and proposed two clustering algorithms CMSPC and CFDPm, which both can detect arbitrary shaped clusters.

CMSPC proposed in our study aims to improve the quality of clustering for the data containing arbitrary shaped clusters. CMSPC is based on similarity between a point and points in a cluster. For two objects with cutoff distance, we consider the degree of membership one object toward the clusters containing the other object and merge temporary clusters which these two objects belong to if the degree of membership meet a certain value. Using CMSPC, Clusters are recognized regardless of their shape, outliers also can be detected during the clustering procedure. The other algorithm proposed in our paper is modified from novel CFDP algorithms. For the dataset with multi-peak cluster, it is difficulty to choose exact cluster center points, and CFDP algorithms can lower the quality of clustering because of the inaccurate choosing of cluster center points. By the systematic and comprehensive reviews of the distance among clusters in clustering results, intra-cluster distance and the impact of merging two clusters on the overall Davies-Bouldin Index, our paper aims to merge clusters conditionally and then improve the clustering quality that lowered by choosing cluster center point in CFDP clustering procedure.

In order to test the effectiveness of two algorithms proposed in our paper, we conducted

clustering experiments on benchmark datasets. The results of experiments show that: First, CMSPC algorithm can detect arbitrary shaped clusters with a high clustering quality and identify outliers. Second, CFDPm algorithm is expected to solve the problem of CFDP algorithm with lowering quality of clustering on dataset with multi-peak cluster by choosing inaccurate cluster center point.

Key words: Arbitrary shaped clusters, Clustering, Spatial data, Similarity measure

目 录

摘要.....	I
Abstract.....	II
第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状.....	3
1.3 本文的主要研究内容.....	4
1.4 论文的组织框架.....	5
第二章 聚类分析概述.....	6
2.1 聚类的相关概念和定义.....	6
2.2 聚类中的距离度量.....	8
2.2.1 对象间的距离.....	8
2.2.2 簇间距离.....	9
2.3 聚类方法.....	10
2.3.1 基于距离的聚类方法.....	10
2.3.2 基于密度的聚类.....	11
2.4 聚类评价指标.....	11
2.4.1 外部评价指标.....	12
2.4.2 内部评价指标 Davies-Bouldin.....	14
2.5 本章小结.....	14
第三章 基于点与簇内多点相似的聚类算法.....	15
3.1 基于点与簇内多点相似算法的提出.....	15
3.2 基于点与簇内多点相似的聚类算法 CMSPC.....	17
3.2.1 相关概念及定义.....	17

3.2.2 基于点与簇内多点相似的聚类算法 CMSPC.....	19
3.2.3 复杂度分析.....	21
3.3 实验及结果分析.....	21
3.3.1 实验环境及对比算法参数设置.....	21
3.3.2 实验所用数据集.....	22
3.3.3 CMSPC 聚类算法实验.....	24
3.4 本章小结.....	29
第四章 改进的聚类算法 CFDPm.....	30
4.1 CFDP 算法介绍.....	30
4.2 CFDPm 算法.....	31
4.2.1 CFDP 算法在多峰簇数据集上的聚类结果分析.....	32
4.2.2 CFDPm 算法思想.....	33
4.2.3 相关定义及概念.....	33
4.2.4 CFDPm 算法详细描述.....	33
4.3 实验及结果分析.....	35
4.3.1 实验数据集.....	35
4.3.2 CFDPm 算法实验.....	36
4.4 本章小结.....	41
第五章 总结与展望.....	42
5.1 本文工作总结.....	42
5.2 未来工作展望.....	42
参考文献.....	44
在学期间的研究成果.....	47
致 谢.....	48

第一章 绪论

1.1 研究背景及意义

随着计算机在各行各业的普及和互联网技术的日益发展,每天有来自科学、工程、商业、社会等各领域数千兆兆字节(PB)的数据产生。大容量存储设备的不断更新使得能存储的数据量也达到了PB级别。并且,每天产生的数据量的增长速度也在不断增大。数据的类型有文本、图片、视频、地理数据、网络数据、时间序列数据等多种形式^[1]。数据的爆炸式增长是大数据时代到来的一个标志。尽管数据的量很大,但是其中蕴藏的知识是极少的。历史数据量的增长,使得数据分析和处理的难度变大了。对大规模数据观察细节是不切实际的。如何使用通用的工具从来自数据库系统、数据仓库、事物数据系统、空间数据系统、互联网等数据源的海量数据中发现有价值的信息变得极其重要。

数据挖掘是在这个时候应运而生的新技术。数据挖掘又叫知识发现,它是能从大量数据中发现和提取有趣模式和知识的一个过程,数据挖掘可以发现数据中隐藏的模式。数据挖掘结合了机器学习、统计学、数据库技术、信息检索等多种技术,如今已被广泛应用到商业、科学研究、政府安全等多个领域。数据挖掘主要包含数据特征化和数据区分、频繁模式挖掘和关联分析、分类和预测、聚类分析这几种功能^[1]。对实体的或者抽象的对象进行分类,是人类认识事物和分析事物的基本方法。作为数据挖掘的基本工具,如今分类和聚类经常被用来做有意义的分类工作。

聚类,又称作聚类分析。在数据挖掘中,聚类将一个数据对象的集合划分成不同的簇,使得簇内的数据对象具有较高的相似性,而不同簇之间的数据对象具有较低的相似性。聚类起源于分类,但又不同于分类。分类是有监督学习,通常的做法是在有类标的数据上进行训练得到一个模型,用得到的分类模型对不含类标的数据进行分类。而聚类在机器学习领域中属于无监督学习,待分析的数据不含类标,数据应该被分成几类以及哪几类,聚类之前都是不清楚的。从这一点来看,聚类比分类难度更大。聚类是一种不需要先验知识的探索性数据分析技术,它既可以被用作单独的分析工具来深入了解数据的分布,也可以作为其他基本数据挖掘问题(如分类、离群点分析)中的一个关键步骤而存在。作为一种最基本的数据挖掘技术,聚类如今已经被广泛应用于模式识别^[2,3]、图像处理^[4-6]、空间数据分析^[7,8]、文本分类和信息检索^[9-12]、市场分析^[13,14]等许多领域。

聚类分析是数据挖掘领域和机器学习领域研究最广泛的技术之一。聚类对于研究者的吸引力在于：对于给定的数据，不需要任何先验知识就能发现数据中的簇或者层级结构。聚类的研究内容有多个方面，包括相似性度量方法的研究、任意形状簇数据的聚类研究、可伸缩聚类算法的研究、高维度数据的聚类、不同类型数据的聚类、处理噪声数据的能力等。研究领域涉及模式识别、图像处理、地理数据分析、文本聚类、社交网络、生物数据等多个领域^[15]。聚类中有诸多因素需要考虑，划分的准则是其中一个需要考虑的因素，即应用场景只需要单个的簇结构还是需要嵌套的分类结构。簇的划分是互斥的还是可以重叠的，若某个数据对象可以属于不同的簇，那就需要模糊聚类。相似性度量也有很多种，需要选取基于距离的度量或者基于连接的度量。从聚类空间上来看，可以是全维度也可以是子空间聚类。对于低维度数据，可以在全空间进行聚类以找出簇结构，但是对于高维数据，在高维空间中发现有意义的簇是困难的。这就需要将高维数据投影在低维空间后在低维子空间进行聚类。聚类有很多要求和挑战，其中最重要的就是聚类质量了。聚类质量的要求是多方面的，如何能在多种类型的数据上保持聚类效果，对于任意形状的数据能否进行有效聚类，对于含有噪声的数据能否保证聚类质量^[15]。其次，随着数据规模的增大，在大规模数据上进行聚类变得越来越重要。可伸缩性是聚类的另一个挑战，这包括四个方面：在全部数据集上进行聚类、对高维数据聚类，对流数据进行聚类和对不同输入顺序的数据聚类的能力。第三个聚类的挑战是有约束的聚类，在实际应用场景中，经常需要考虑现实世界中包括客观条件和人的因素在内的约束条件，如何发现既满足特定约束同时又具有良好聚类特性的簇结构是一项具有挑战的任务。对于聚类来说，聚类的结果最好是可解释、可理解和可用的^[1]。这是聚类面临的另外一个挑战：可解释性和可用性。

聚类出现之后的几十年中，出现了许多聚类算法以满足聚类的不同要求和挑战。按照聚类算法中基于的不同技术划分，聚类算法包括基于距离的聚类算法、基于密度的聚类算法、基于网格的聚类算法、基于概率和生成模型的聚类算法、子空间聚类、高维度聚类、可伸缩聚类。

空间聚类作为一种常用的分析工具是疫情监测、空间流行病学、群体遗传学、园林生态、犯罪行为分析等领域^[16]的主要手段之一。空间聚类是将聚类应用于空间数据的分析中，是空间数据挖掘中的一个主要技术。它是将空间数据进行分簇，使得簇内的空间对象相似性高，簇间的空间对象之间相似性低。地理信息系统（GIS）、包含有医学图像的生物医学应用、农业科学等领域^[17]产生出了大量的空间数据。空间数据是由点、线、曲面、三维几何体、或包含有时间信息的更高维的空间部件构成。在现实世界中，高速路、河流、湖泊等障碍物的存在会影响

聚类结果。空间数据中包含大量的不规则形状的簇，这对传统的数据挖掘算法提出了挑战。由于空间数据量的快速增长，空间聚类受到越来越多研究者的关注和研究，对包含任意形状簇的数据的聚类方法研究是其中一个研究热点。

1.2 国内外研究现状

聚类作为统计学的一个分支，早在数据挖掘出现之前就被开始研究了。划分方法是较早出现的聚类方法，对于给定的参数 k ，划分方法通过迭代再分配技术将数据对象集合划分为 k 个簇，使得每个数据对象距离簇中心（或一个概率分布）的总体偏差最小。在不同的划分方法中有不同的偏差计算函数。K-Means^[18]、K-Medoids^[19]、和 EM(Expectation Maximization)^[20]是具有代表性的划分方法。和通常的划分方法不同，EM 使用了概率分布的概念，使得每个对象都可能属于多个簇，它是一种模糊划分技术。为了把基于中心点的算法应用于大型数据集，Kaufman 提出了 CLARA^[21]算法，该算法会抽取数据集的一个样本，在小样本上使用 PAM (Partitioning Around Medoids) 方法产生中心点，这样能提高算法的效率。然而，对样本的不同抽取方式使得 CLARA 的聚类质量变化较大。为了保证在大型数据集上的聚类质量，Ng 和 Han 于 1994 年提出的 CLARANS^[22]在选取中心点的时候采用随机搜索，这样既能保证较好的聚类质量，又具有可伸缩性。有些应用需要获取不同级别的簇结构，划分方法不能满足这个要求，这个时候就出现了层次聚类方法。层次聚类通过自上而下分裂或者自下而上合并的过程产生一个嵌套的树形结构，每一层都是一个划分。AGNES^[21]和 DIANA^[21]是较早出现的层次聚类方法，因为它们使用过于简单的度量来进行分裂或合并，而分裂或合并过程又不可逆，使得算法容易产生错误的簇结构。为了提高层次聚类效果，BIRCH^[23]、CURE^[24]、Chameleon^[25]通过将层次聚类和其他聚类方法结合，在底层节点上使用其他聚类方法，这种做法可以提高层次聚类的质量。BIRCH 首次提出的聚类特征 (Clustering Feature) 和聚类特征树 (CF Tree) 的概念被后来的研究广泛采用。划分方法和层次方法多是基于对象之间的距离，这类方法适用于球形数据，很难发现任意形状的簇。Ester 于 1996 年提出的 DBSCAN 是一种基于密度的聚类方法，它将簇看成是数据空间中被稀疏区域分隔开的稠密区域。该算法能识别出任意形状的簇，且能够识别出噪声点。但是，DBSCAN 需要两个输入参数： ϵ 和 $MinPts$ 来定义密度，在实际应用中通常需要使用多组参数运行多次才能得到好的聚类结果。为了克服 DBSCAN 的这个缺点，Ankerst 等人提出了 OPTICS^[26]算法。OPTICS 算法不会显式地生成簇结构，只需输入一对参数 ϵ 和 $MinPts$ 便能生成一个有序的对象列表，用户可以根据这个对象列表生成不同粒

度的簇结构。为了改善 DBSCAN 和 OPTICS 算法中密度对于输入参数的敏感性, DENCLUE^[27] 算法引入了核密度估计, 基于一组密度分布函数来进行聚类。DENCLUE 在降低噪声对聚类结果的影响方面效果显著。基于距离的方法和基于密度的方法都是以数据为驱动进行聚类, 这种聚类算法的运行时间依赖于数据对象集合的大小。Wang、Yang 和 Muntz 提出了一种空间驱动的聚类方法——STING^[28], 该方法是一种基于网格的多分辨率聚类算法。它把数据空间划分为一个个小单元, 形成层次结构。聚类过程是在这个层次结构上进行。该算法聚类速度和聚类效果取决于网格的最小粒度。对于维数在 10 个以上的数据, 在所有维度构成的空间上发现簇是比较困难的。这时, 在由其中一些属性组成的子空间中进行聚类更可能发现有意义的簇。Agrawal 提出的 CLIQUE^[29] 是一种基于网格的子空间聚类方法, 该方法能发现任意形状的基于密度的簇。高斯混合模型(GMM) 是一种基于概率和生成模型的聚类方法, 在 GMM 中, 数据集被假定为由若干参数不同的高斯分布组合而成, 使用 EM 方法对模型的参数进行估计, 好的划分被认为具有高的拟合度。Wehrens、Tran^[30] 于 2006 年提出的新的基于密度的聚类算法以及 Borries 和 Wang^[31] 于 2009 提出的新的划分方法主要用来解决高维数据的聚类问题。

在对包含任意形状簇的数据的聚类方法研究中, 有众多新的聚类算法涌现出来。CFDP^[32] (Clustering by Fast search and find of Density Peaks) 是其中之一, 它是 Alex 等人于 2014 年发表在《科学》上的一种新的能发现任意形状簇的聚类算法。该算法将局部密度最大的对象看作是簇中心, 它认为簇的中心就是那些比周围的对象具有更高密度、距离比其密度更大的对象较远的点。在确定簇中心后, 剩余对象被按照一定的规则分配给不同的簇。该算法的优点是能识别任意形状、不同分布密度的簇, 同时能自动识别出离群点。ABACUS^[33] 是一种基于形状的聚类算法, 该算法第一步先通过一个迭代过程发现数据的轮廓, 第二步将剩余的数据对象分配到轮廓主干上面, 该算法具有较好的伸缩性, 能够用于大数据集。Liu、Zhou 等人于 2013 年提出的基于路径的聚类算法^[34] 使用改进的 Floyd 算法来降低路径距离计算的时间复杂度。这种做法使得改进后的算法能在包含有任意形状的数据集中高效聚类。Galluccio 等人^[35] 提出了一种新的基于双根最小生成树的相似性度量, 将这个相似性度量和谱聚类方法结合起来在非球形簇的数据集上能够取得较好的聚类效果。

1.3 本文的主要研究内容

空间数据的一个重要特性就是数据集中包含大量的不规则形状的簇, 含有异

常点。然而，专门针对挖掘任意形状簇的聚类算法研究并不多。本文主要对包含任意形状簇的数据的聚类算法进行研究，提出两个聚类算法 CMSPC 和 CFDPm。

(1) CMSPC: 旨在对包含有任意形状簇的数据进行聚类，提高聚类质量。CMSPC 使用点与簇内多点相似的思想，引入点关于簇的归属度的概念来评价对象属于簇的可能性。初始状态下，每一个对象被看做是一个单独的簇。对于数据集中相似性在一定阈值之内的两个对象连接一条边。对于边两端的顶点，分别计算顶点对象关于另一端对象所在簇的归属度，对于满足一定归属度的边，合并两个端点所在的簇，最终形成簇结构，同时识别出异常点。

(2) CFDPm: 对一种新型聚类算法 CFDP 进行了改进，在此基础上提出新的聚类算法 CFDPm。对 CFDP 算法因簇中心的选取带来的聚类质量的不稳定性问题进行改进，通过引入聚类内部评价指标，结合 CFDP 算法聚类结果的平均距离、簇间距离，对距离较近、比较紧凑的两个簇进行合并。从而改善 CFDP 算法在簇中心选择过多时的聚类质量。

1.4 论文的组织框架

本论文一共分为 5 章，主要包含以下内容：

第一章 绪论。主要介绍聚类研究背景和意义，简单叙述了国内外聚类分析的研究现状。在 1.3 节介绍了本论文的主要研究内容。最后给出了本文的组织框架。

第二章 聚类分析的相关概念及术语。首先对聚类的定义进行了阐述。之后介绍了聚类中的距离度量，分别介绍了对象间的距离度量和簇间的距离。接下来介绍了两类聚类算法。最后，介绍聚类的评价指标，包括内部评价指标和外部评价指标，并对外部评价指标的优劣进行了对比和分析。

第三章 基于点与簇内多点相似的聚类算法。对本文提出的聚类算法 CMSPC 进行了详细介绍，并在多个数据集上进行实验来验证聚类的有效性和聚类质量。

第四章 改进的聚类算法 CFDPm。对一种新型的聚类算法 CFDP 进行了改进，主要针对该算法因簇中心的选取不准确所带来的聚类质量不稳定问题，通过合并簇的方式来进行改善。在多个数据集上对改进后的算法的聚类质量、稳定性、时间性能方面进行了实验验证。

第五章 全文总结及研究展望。对本文所做的研究工作做了回顾和总结，并展望了下个阶段的研究方向。

第二章 聚类分析概述

为方便本论文的论述,本章对聚类的基本概念和知识以及本文第三章和第四章所涉及的一些概念进行了介绍。包括聚类的概念和定义、聚类中用到的距离度量、两类常用的聚类算法、聚类的评价指标等。

2.1 聚类的相关概念和定义

聚类分析(Cluster Analysis)简称聚类,它是根据数据对象本身和对象之间的描述信息来将数据对象集划分为不同的组或者子集,使得组内的对象相似度高,而不同组的对象间相似度低。聚类在学术界没有一个统一的定义。Everitt^[36]于1981年将聚类描述为:“给定一个对象或个体的集合,其中每一个对象(或个体)用一组数值度量来描述,制定一个分类方案将对象分成若干组,使得组内的对象之间相似,而不同组之间的对象之间不相似,组的个数以及每个组的特征需要确定下来。”Bacher^[37]于1995年对聚类的简介描述:“找到 k 个簇(或者由 k 个簇组成的划分)使得同一簇中的对象间相似,而不同簇之间的对象不相似。”对于聚类的定义,尽管有几种不同的聚类的定义,但是它们之间差异并不大,有两点基本是确定的:第一,簇的个数在聚类之后是确定的;第二,簇内的对象之间应该比簇间对象之间更相似。

$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,m} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n,1} & x_{n,2} & \cdots & x_{n,m} \end{bmatrix} \quad (2-1)$$

数据对象的集合简称**数据集**。数据对象也可以称作样本、对象、点。数据对象通过属性来描述。本文用到的数据集的属性有标称属性和数值属性两种。样本量为 n ,属性个数为 m 的数据集 X 通常用一个矩阵来表示,如式2-1。矩阵 X 中,每一行表示由 m 个属性组成的一个数据对象 $x_i(i \in \{1,2,\cdots,n\})$ 。式中 $x_{i,j}$ 用来表示数据集 X 中第 i 个元素的第 j 个属性值,其中 $i \in \{1,2,\cdots,n\}, j \in \{1,2,\cdots,m\}$ 。在数据集 X 之上,本文对聚类的定义如定义2-1所示。

定义2-1: 聚类。对于给定数据集 $X = \{x_1, x_2, \cdots, x_n\}$,将 X 划分为 k 个不相交的子集 $C_1, C_2, \cdots, C_k$ 的过程,叫做聚类,其中 $\forall i \in \{1,2,\cdots,k\}: C_i \subset X \wedge X = C_1 \cup C_2 \cup \cdots \cup C_k \wedge C_1 \cap C_2 \cap \cdots \cap C_k = \emptyset$ 。

在定义 2-1 中, 聚类得到的子集 $C_1, C_2, \dots, C_k$ 叫做簇。以这些簇为元素组成的集合 $C = \{C_1, C_2, \dots, C_k\}$ 被称为 X 的一个划分。将 X 中属于同一簇的数据对象赋予同样的一个字符标记, 这样对于 $x_1, x_2, \dots, x_n$, 可以得到相对应的标记 $p_1, p_2, \dots, p_n$, $p_1, p_2, \dots, p_n$ 分别被称作数据对象 $x_1, x_2, \dots, x_n$ 的聚类类标。 $p_1, p_2, \dots, p_n$ 组成的集合 P 也被称为数据集 X 的聚类类标。对于数据集 X , 若已知每一个数据对象的类别归属 $y_1, y_2, \dots, y_n$, 则 $y_1, y_2, \dots, y_n$ 分别被称作数据对象 $x_1, x_2, \dots, x_n$ 的真实类标, 集合 $Y = \{y_1, y_2, \dots, y_n\}$ 被称作数据集 X 的真实类标。

从定义 2-1 来看, 存在很多满足条件的划分, 但是大多数划分都是无意义的, 聚类分析就是要找出符合数据集真实分布的划分。

簇是相似对象的一个集合。但是, 簇有时候不容易定义。如图 2-1 中, a 是数据对象集合的 2 维图, 图中标志 'x' 代表数据对象。图中 b 是数据对象集的一个划分, 具有同样的数字标记的对象属于同一个簇, 图中一共有 7 个簇。然而, 存在其他的划分。 $\{1, 2\}$ 、 $\{3, 4, 5\}$ 、 $\{6, 7\}$ 表示标记为 1、2 的对象属于同一簇, 标记为 3、4、5 的对象属于同一簇, 标记为 6、7 的对象属于同一簇, 这种划分比图 b 中的划分粒度较粗, 将数据集看做看作 3 个大簇。 $\{1\}$ 、 $\{2\}$ 、 $\{3, 4, 5\}$ 、 $\{6\}$ 、 $\{7\}$ 又是另一种划分方式。可见, 簇的定义并没有一个绝对的标准, 按照不同的标准, 数据集可以有不同的划分。聚类旨在找出有意义的簇, 然而, 什么样的簇是有意义的, 它取决于数据本身的分布和用户聚类分析的目标。

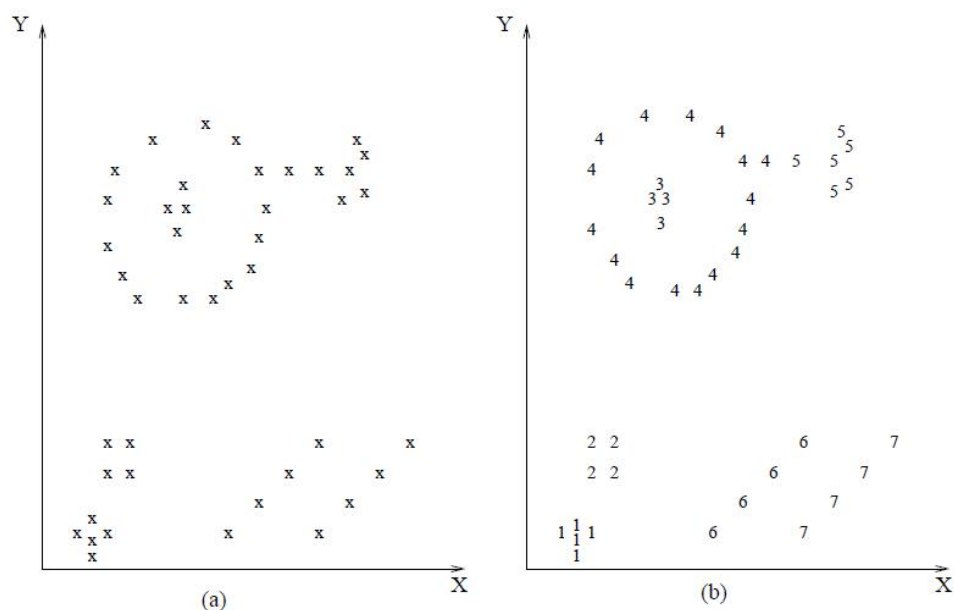


图 2-1 聚类示意图^[38]

聚类分析通常包括这几个步骤: 数据预处理、特征提取和特征选择、定义相似性或者距离度量、聚类过程、聚类结果评价。特征选取、距离度量定义、聚类

算法都会影响最终的聚类质量。本文主要研究聚类算法，本文中“聚类”这个词用来指代聚类过程这个步骤或者聚类算法。距离度量、聚类方法和聚类结果的评价将在本章后续章节中进行介绍。

2.2 聚类中的距离度量

聚类的目标是使得簇内的数据对象之间相似，不同簇之间的数据对象相异。相似性描述来自于距离度量，距离用来表示两个数据对象之间的远离程度。两个对象越接近，则其距离越小，相似度越高；反之，两个对象越不相似，则其距离越大，相似度越低。对于样本大小为 n 、属性个数为 m 的数据集 X ，所有对象两两之间的距离可以构成一个 $n \times n$ 的距离矩阵，该矩阵用 $dist_matrix(X)$ 来表示，形式如公式 2-2 所示。

$$dist_matrix(X) = \begin{bmatrix} d(x_1, x_1) & d(x_1, x_2) & \cdots & d(x_1, x_n) \\ d(x_2, x_1) & d(x_2, x_2) & \cdots & d(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ d(x_n, x_1) & d(x_n, x_2) & \cdots & d(x_n, x_n) \end{bmatrix} \quad (2-2)$$

式中： $d(x_i, x_j)$ 在下文中介绍，它是一个计算距离的函数，返回值是对象 x_i 和 x_j 之间的距离。矩阵 $dist_matrix(X)$ 对角线上的元素都为0，用来表示对象和它本身之间的距离，对象和自身之间的距离为0。可以通过矩阵的行下标和列下标来引用矩阵中的元素， $dist_matrix[i, j]$ 用来表示矩阵 $dist_matrix$ 中的第 i 行第 j 列的元素 $d(x_i, x_j)$ 。

2.2.1 对象间的距离

两个数据对象 x_i 和 x_j 之间的距离用函数 $d(x_i, x_j)$ 来计算，可以用 $d_{i,j}$ 来表示。下文列出了几种常见的距离度量方式。其中欧式距离、曼哈顿距离、切比雪夫距离都属于闵可夫斯基距离，也分别被称作 L_2 范数、 L_1 范数、 L_∞ 范数。不同的距离度量有不同的适用场景，比如，余弦距离可以用于高维数据计算距离。

(1) 欧式距离：

$$d(x_i, x_j) = \sqrt{\sum_{q=1}^m (x_{i,q} - x_{j,q})^2} \quad (2-3)$$

(2) 曼哈顿距离：

$$d(x_i, x_j) = \sum_{q=1}^m |x_{i,q} - x_{j,q}| \quad (2-4)$$

(3) 余弦距离:

$$d(x_i, x_j) = 1 - \frac{x_i \cdot x_j}{\|x_i\| \|x_j\|} \quad (2-5)$$

(4) 马氏距离:

$$d(x_i, x_j) = \sqrt{(x_i - x_j)^T S^{-1} (x_i - x_j)} \quad (2-6)$$

式中: S^{-1} 为数据集 X 的协方差矩阵。

(5) 切比雪夫距离:

$$d(x_i, x_j) = \max_{q=1}^m (|x_{i,q} - x_{j,q}|) \quad (2-7)$$

2.2.2 簇间距离

有些聚类算法需要计算两个簇之间的距离, 比如凝聚层次聚类在合并簇的时候需要计算簇之间的距离, 在凝聚的层次聚类中, 簇间距离度量方法也称作连接方法。5 种常用的簇间距离度量包括最小距离、最大距离、均值距离、平均距离、Ward 距离。

(1) 最小距离:

$$d(C_i, C_j) = \min_{x \in C_i, x' \in C_j} |x - x'| \quad (2-8)$$

(2) 最大距离:

$$d(C_i, C_j) = \max_{x \in C_i, x' \in C_j} |x - x'| \quad (2-9)$$

(3) 均值距离:

$$d(C_i, C_j) = |\mu_i - \mu_j| \quad (2-10)$$

式中: μ_i 表示簇 C_i 的均值。

(4) 平均距离:

$$d(C_i, C_j) = \frac{1}{|C_i| |C_j|} \sum_{x \in C_i, x' \in C_j} |x - x'| \quad (2-11)$$

式中: $|C_i|$ 表示簇 C_i 中样本个数。

(5) Ward 距离:

$$d(C_i, C_j) = \text{dist}(x, \mu_{C_i \cup C_j})^2 \quad (2-12)$$

式中: $\mu_{C_i \cup C_j}$ 表示 C_i 和 C_j 合并后的均值。

2.3 聚类方法

本节介绍了两类典型的聚类方法：基于距离的聚类方法和基于密度的聚类方法。

2.3.1 基于距离的聚类方法

基于距离的聚类方法需要定义两个对象之间的距离度量。基于距离的聚类找到一个划分使得簇内的对象之间距离尽可能小，不同簇的对象之间距离尽可能大。划分方法和层次聚类都属于基于距离的聚类。

(1) 划分方法

划分聚类方法在数据挖掘出现之前就开始流行。划分方法的思想是：对于给定数据集 X 和输入参数 k ，通过迭代再分配技术将数据集划分为 k 个簇 $C_1, C_2, \dots, C_k$ ，使得数据对象距离簇中心（或一个概率分布）的总体误差最小。不同的划分聚类方法具有不同的误差计算函数。划分方法中比较著名的是 K-Means^[18]（K-均值）。该算法首先初始化 k 个簇中心，将 X 中所有数据对象分配给 k 个中心中距其最近的一个中心，通过计算均值，产生新的中心点，对其他数据对象重新划分并计算新的中心点。如此迭代，直至簇结构不再变化为止。这种通过一步步迭代将数据对象重新分配到各簇来改进划分的过程被称为迭代的再分配。K-Means 的总体误差 E 用误差平方和来定义，形式如 2-13 所示。

$$E = \sum_{i=1}^k \sum_{x \in C_i} dist(x, \mu_i)^2 \quad (2-13)$$

式 2-13 中： μ_i 表示簇 C_i 的形心，用来代表簇 C_i ，通过计算 C_i 中所有对象的均值来得到。 $dist(x, \mu_i)$ 表示对象 x 和 μ_i 之差，在这里指代 x 和 μ_i 之间的欧式距离。

K-Means 对初始簇中心点的选取比较敏感，若初始中心点选得好，聚类效果可能更好，也能加速收敛过程。在 K-Means 算法中，初始中心点的选取十分重要。Arthur 等人于 2007 年提出的 K-Means++^[39] 算法提供了一种初始簇中心点选取方法，它在选取初始簇中心时选择 k 个距离尽可能远的点作为初始簇中心点，这种选取方式能选出合适的簇中心点。

(2) 层次聚类

层次聚类方法可以根据不同粒度给出簇划分的一个树形层次结构。按照聚类过程中树形结构的扩展方向分两种：分裂的层次聚类、凝聚的层次聚类。在凝聚的层次聚类中，初始状态下，每个对象被看做是一个簇，通过一个迭代过程把簇

合并为更大的簇，直到所有对象都属于一个簇，形成一个树形结构，其中的节点是簇，每一层都是一个划分。合并方法是：在每一次合并中，合并最相似的两个簇。分裂的聚类方法是将整个数据集看作是一个簇（即树根），用迭代的方法将这个簇划分为每个簇只有一个节点的簇划分。这两种策略都能形成一个完整的树形结构，用户也可以根据实际应用需求来人为规定聚类终止条件。AGNES 是凝聚的层次聚类算法，DIANA 是分裂的层次聚类算法。BIRCH 是一种使用了聚类特征树（Clustering Feature Tree）的多阶段层次聚类，该算法使用由簇统计量组成的聚类特征来描述一个簇，这种方法时间复杂度小，适用于大规模数据集。在凝聚层次聚类中，可以使用不同的簇间距离来对簇之间的相似性进行度量，这样便产生出不同的凝聚层次聚类算法。本文第三章的对比算法中使用了组平均方法和 Ward 方法。簇间距离在本章第 2 节有所介绍。

组平均方法：簇间距离中的平均距离在计算两个簇之间的距离时使用两个簇的所有对象间距离的平均值作为簇间距离，这种方法比质心算法更能避免离群点对聚类结果的影响。这种方法也称作 Average linkage 聚类，本文中简写为 ALC。

Ward 方法：使用 Ward 距离来作为簇间距离的聚类方法叫做 Ward linkage 聚类，本文中简写为 WLC。这种方法和 K-Means 类似，使用合并两簇所带来的离差平方和 ESS（Error Sum of Squares）作为两个簇之间的距离。

2.3.2 基于密度的聚类

和基于距离的聚类方法使用距离来划分簇不同，基于密度的聚类算法把簇看成数据空间中被稀疏区域分割开的稠密区域，使用密度作为基准来进行簇的扩张，当对象邻域的密度大于某个阈值的时候，将领域扩张为簇的一部分，可以发现任意形状的簇。DBSCAN 算法有两个输入参数 ϵ 和 $MinPts$ 。 ϵ 定义对象的邻域半径，对象的密度通过以 ϵ 为半径的邻域内对象的个数来度量。 $MinPts$ 是用户输入的用来指定稠密区域密度阈值。如果一个对象的 ϵ 邻域内的对象个数不小于 $MinPts$ ，则被称为核心对象，核心对象是稠密区域的主干部分。DBSCAN 通过密度相连的概念把邻域内密度大于 $MinPts$ 的数据对象扩张到核心对象。 ϵ 邻域内对象个数少于 $MinPts$ 的对象密度很小，被认为是噪声点。DBSCAN, OPTICS, DENCLUE 和 CLIQUE 是几种比较常用的基于密度的聚类算法。这几种方法在具体操作上互不相同，也有众多的变种算法。

2.4 聚类评价指标

Jain 和 Dubes 在《Algorithms for Clustering Data》^[40] 一书中说过，“聚类结

果的有效性验证是聚类分析中非常困难和令人挫败的一个任务。如果没有在这方面坚持不懈的努力，聚类分析将永远只是那些相信自己有经验并且有足够勇气的人的魔法。”聚类结果的评价是任何聚类分析必不可少的一步。聚类的评价指标有内部标准（Internal Criteria）和外部标准（External Criteria）两种。内部标准是用数据集中的数据对象间的关系来对聚类结果进行评价，有一种内部评价是使用聚类结果各簇的误差平方和作为评价标准。外部标准是通过已知的簇结构来对聚类结果进行评价。若应用中已知数据的真实类标信息，则使用外部评价指标对聚类结果的进行评价更为准确。在外部评价指标中有众多评价指标可以选择，为了测试哪个评价指标更好，我们在随机划分上做了实验，实验结果显示评价指标ARI在避免受随机划分影响方面表现最好。

2.4.1 外部评价指标

(1) ARI (Adjusted Rand Index)

RI (Rand Index) 是统计学中的一个概念，数据挖掘中经常用来对分类或聚类结果进行评价。在聚类评价中，用来表示聚类结果和真实簇结构的相似程度。ARI^[41]是对RI的修正，避免了RI评价中对于随机划分不够准确的现象，它是Hubert和Arabie于1985年提出的评价指标，如今已被许多分类和聚类的研究者所采用，其公式如2-14所示。

$$ARI = \frac{\sum_{i=1}^k \sum_{j=1}^{k'} \binom{n_{i,j}}{2} - t_3}{\frac{1}{2}(t_1 + t_2) - t_3} \quad (2-14)$$

$$t_1 = \sum_{i=1}^k \binom{|C_i|}{2} \quad (2-15)$$

$$t_2 = \sum_{j=1}^{k'} \binom{|C'_j|}{2} \quad (2-16)$$

$$t_3 = \frac{2t_1 t_2}{n(n-1)} \quad (2-17)$$

$$\binom{n_{i,j}}{2} \quad (2-18)$$

式2-14中： t_1 、 t_2 、 t_3 分别如公式2-15、2-16、2-17所示。其中 C 和 C' 是两个划分，分别包含 k 、 k' 个簇， C_i 和 C'_j 分别为 C 、 C' 中的簇。 $n_{i,j}$ 表示同时在簇 C_i 和簇 C'_j 中的数据对象的个数，也可以表示为 $|C_i \cap C'_j|$ 。式2-18表示不考

虑顺序的排列组合，表示从 $n_{i,j}$ 个不同的元素中抽取两个的组合次数。

除 ARI 外，常用的聚类评价指标还有 MI、NMI、V-measure。下文中我们通过两个实验来测试这几种评价指标的优劣。

设样本总量为 100，我们独立随机生成包含两个簇的两个划分，计算他们的 4 种聚类评价指标，这样运行 5 次，取各评价指标的平均值作为相应评价指标的得分值。取簇个数为 12、23、35、45、56、67、78、89、100，同样随机生成两个划分，计算评价指标，同样运行 5 次，取平均值。将聚类指标画出带有误差棒的折线图，如图 2-2 (a) 所示，横坐标是不同的簇个数，纵坐标是各评价指标的值。从图中可以看出随着类个数不断增大到 100 时，NMI、V-measure 值都逐渐增大至 0.82，MI 更是在类个数增大至 18 时，其得分值就变为 1 了，这显然是不合理的。而 ARI 一直保持在 0 附近。可见，对于两个随机划分，随着簇个数不断增大至样本总量，ARI 评价指标始终能保持为 0，而 MI、NMI、V-measure 会随着类个数增大得分值逐渐增大。

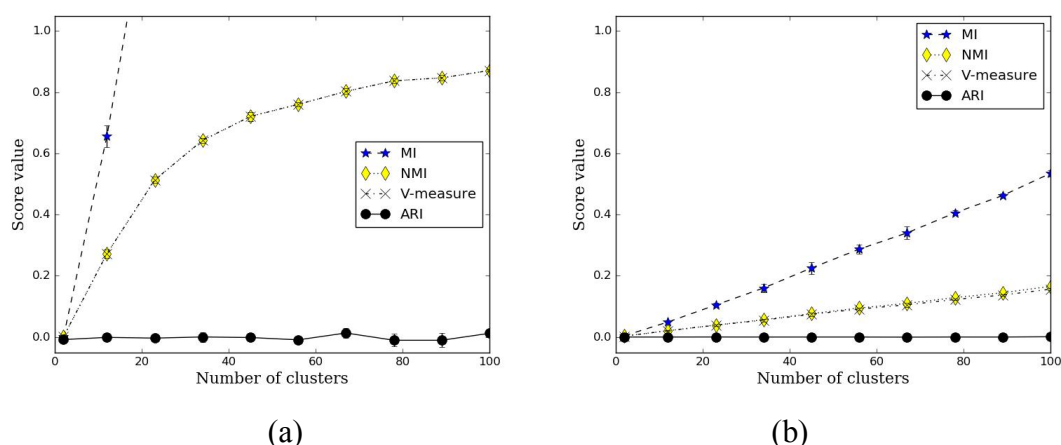


图 2-2 聚类评价指标在随机划分中的表现

设样本总量为 1000，对于每个评价指标，随机生成一个包含 10 个类的划分 C ，再随机生成包含 2、12、23、35、45、56、67、78、89、100 个簇的划分 C_1 、 C_2 、 C_3 、 C_4 、 C_5 、 C_6 、 C_7 、 C_8 、 C_9 、 C_{10} ，对于每个簇个数都执行 5 次，计算 C 和 $C_i, i \in \{1, 2, \dots, 10\}$ 的各评价指标，取 5 次执行结果的平均值，画出带有误差棒的折线图，如图 2-2 (b) 所示。由图可以看出，随着类个数不断增大到 100，NMI、V-measure 的得分都逐渐升高至 0.18，而 MI 得分值在类个数为 100 的时候增大到了 0.5，只有 ARI 的得分值保持在 0 附近。

由上面两个实验可以得出：ARI 不受随机划分的影响，也不受数据集类个数的影响，在数据集的类个数很大的情况下，也能对聚类效果进行合理评价。使用 ARI 对聚类结果进行评价更加准确。本文实验中使用 ARI 对聚类结果进行评价。

2.4.2 内部评价指标 Davies-Bouldin

Davies-Bouldin 指标是 Davies 和 Bouldin 于 1979 年提出的一种聚类结果评价方法^[42]，它是一种内部评价指标，简称 DBI。DBI 的定义如式 2-23 所示。

$$S_{C_i} = \frac{1}{|C_i|} \sum_{x \in C_i} \|x - \mu_{C_i}\| \quad (2-19)$$

$$M_{C_i, C_j} = \|u_{C_i} - u_{C_j}\|_p \quad (2-20)$$

$$R_{C_i, C_j} = \frac{S_{C_i} + S_{C_j}}{M_{C_i, C_j}} \quad (2-21)$$

$$D_{C_i} = \max_{C_i \neq C_j} R_{C_i, C_j} \quad (2-22)$$

$$DB = \frac{1}{k} \sum_{C_i \in C} D_{C_i} \quad (2-23)$$

式 2-19、2-20、2-21、2-22、2-23 中： $C = \{C_1, C_2, \dots, C_k\}$ 是数据集 X 的 1 个划分，共包含 k 个簇， C_i 表示第 i 个簇。数据集 X 是 $n \times m$ 的一个矩阵。 n 是 X 的样本总量， m 是数据的维数。 u_{C_i} 是簇 C_i 的中心，可以用均值表示。 p 是范数，当 p 为 2 时，表示 2 范数，也就是欧式距离。 S_{C_i} 用来评价簇 C_i 内的离散程度，而 M_{C_i, C_j} 用来描述簇 C_i 和 C_j 之间的分离程度。DBI 使用簇内离散程度和簇间距离来评价聚类结果的优劣，簇内越紧凑，则簇间距离越大，DBI 值越小，聚类结果也就越好。和其他常用的聚类内部评价指标如 Dunn's Index、Silhouette Index 相比，DBI 对边界点不敏感。

2.5 本章小结

本章主要介绍了聚类的相关概念，包括聚类的基本概念，聚类分析中用到的距离度量，常用的两类聚类算法，以及聚类的内部评价指标 DBI 和外部评价指标。

第三章 基于点与簇内多点相似的聚类算法

本章提出了一个基于点与簇内多点相似的聚类算法。该算法对满足截断距离的两对象 x_i 和 x_j ，计算 x_i 关于临时簇 $C(x_j)$ 的归属度和 x_j 关于临时簇 $C(x_i)$ 的归属度，当满足一定归属度时合并簇 $C(x_i)$ 和 $C(x_j)$ 。本章引入了点关于簇的归属度概念，对提出的新算法 CMSPC 进行了详细描述，之后在多个基准数据集上对聚类的有效性和聚类的质量进行了实验。

3.1 基于点与簇内多点相似算法的提出

聚类分析经过多年发展产生了许多聚类算法，本文第二章 2.3 节介绍了其中的一些具有代表性的聚类算法。不同的聚类算法被设计来解决不同的问题，很难找出一个能适应各种应用场景的通用聚类算法。本文主要进行对任意形状簇的聚类算法研究。挖掘任意形状簇的聚类算法被用于空间挖掘、图形分割等领域。在空间数据中，数据并非是由紧凑的球形簇构成，而是包含有不同形状、不同大小的簇，也会包含有异常点和噪声点。现有的聚类算法中，能处理任意形状簇的聚类算法并不多，有三类聚类方法可以用于任意形状簇的发现，包括基于密度的聚类、基于图论的聚类、基于数据收缩的聚类算法。基于密度的算法通过定义局部密度可以发现任意形状的簇，同时识别出异常点。DBSCAN 是基于密度的聚类算法，需要两个输入参数 ε 和 $MinPts$ 来定义密度，聚类结果受这两个参数的影响比较大，实际应用中很难确定这两个参数的搭配。谱聚类是一种基于图论的方法，它将数据对象看做顶点，两个数据对象的相似度作为顶点之间的带权边，通过去掉权重小的边将数据集分割成 k 个子图，也就是 k 个簇。谱聚类可以识别出任意形状的簇，并且抗噪能力较好。然而，谱聚类需要输入类个数 k 作为输入参数。在实际应用中类个数通常难以获得，空间数据的复杂性和庞大规模使得类个数的获得相当困难。基于数据收缩的方法通过抽取数据集的一部分样本来代表总体样本，先确定簇的主干，再将剩余样本分配到主干之上。但是自定义的收缩率有可能让簇变形，并且收缩幅度过大容易造成聚类性能的下降。本文提出了一种基于点与簇内多点相似的聚类算法 CMSPC，该算法聚类时与簇的形状和簇大小无关，同时能识别出异常点。CMSPC 初始时将每一个对象点看做是一个单独的簇，通过考虑两两对象的相似度，对满足一定点与簇相似度的两个簇进行合并。

聚类是将数据集划分为多个簇，使得簇内的元素比簇间的元素具有更高的相

似性。相似性最直观的度量就是距离，两个对象距离越小越相似，距离越大越不相似。距离小到哪种程度就可以认为是一个簇，需要有一个界限来划分，在本章的研究中，这个界限也可以叫做截断距离。小于截断距离的两个点属于同一个簇，大于截断距离的两个点不属于同一个簇。但是仅仅依据这个截断距离来合并两个对象的方案是不妥的，因为这种合并会受到噪声点的干扰。这种合并方案在另外一种情况下也不适用，当两个簇被一个桥连接起来的时候，这种合并方案会因为桥的存在而将两个簇合并为一个簇。简单得使用截断距离来合并簇只适用于簇内紧凑，簇间分离，密度均匀的数据集，这种简单的合并策略在复杂的空间数据上会失效。本章中提出的 CMSPC 聚类算法不仅考虑两个对象之间的距离，还考虑了其中一个对象和另一对象所在的临时簇中其他对象的相似度。

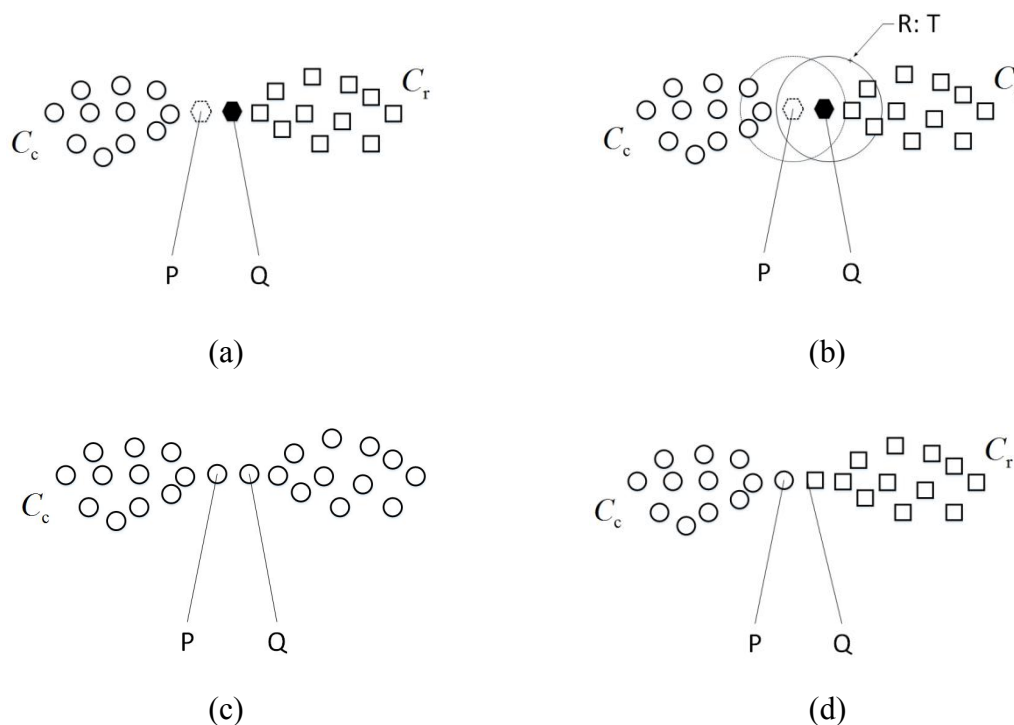


图 3-1 点与簇内多点相似示意图

在图 3-1(a)中，有两个簇 C_c 和 C_r ，其对象分别用空心圆和空心矩形表示。有两个点 P 和 Q，分别用空心六边形和实心六边形表示，这两个点都是单点簇。现在对这四个簇进行合并。如(b)所示，两个圆的半径长度为截断距离 T，这两个圆的圆心分别是 P 和 Q。假如考虑对两个簇最小距离小于截断距离的簇进行合并，则首先 P 和簇 C_c 合并，Q 和簇 C_r 合并。P 和 Q 又在截断距离之内，这时又把簇 C_c 和 C_r 合并，最终得到如图中(c)所示的簇结构，只包含一个簇。这就是只考虑点与点的相似性造成的结果。在本章介绍的 CMSPC 算法中，我们考虑了点与簇内多点的相似性。在合并簇时，当点和簇内的至少 3 个点的距离在截断距离 T 之内

时, 才会合并簇和点所在簇。这样, 如图中(b)所示, 以 P 为圆心, 截断距离 T 为半径的圆中有 3 个点在簇 C_c 中, 于是合并 P 所在簇和 C_c 。同样, Q 所在簇会被合并到簇 C_r 。最终得到如图中(d)所示的簇结构, P 和 C_c 在同一簇, Q 和 C_r 在同一簇。这种划分是合理的。在合并簇时, 我们对小于截断距离的两个点, 考虑点和另一点所在簇中多点的相似度。对于和任何点的距离都大于截断距离的点, 我们就可以把它当做是异常点。同时, 这种合并簇的方式与簇的形状和簇的大小无关。按照这样的思路来设计聚类算法可以满足发现任意簇、识别异常点、与簇大小无关的聚类要求, 接下来一节将详细描述本文提出的聚类算法 CMSPC。

3.2 基于点与簇内多点相似的聚类算法 CMSPC

本文提出的基于点与簇内多点相似的聚类算法 (Clustering by Measuring Similarity between Point and points in Cluster) 是一种合并算法, 该算法遍历数据集中所有对象, 在遍历到每一个对象的时候, 将它所在的簇和距它较近的满足条件的对象所在的簇合并。

3.2.1 相关概念及定义

本章提出的 CMSPC 聚类算法初始状态把每一个对象看做一个单独的簇, 聚类过程中会逐渐合并各簇, 通过考虑对象和簇的关系来决定是否应该合并数据对象所在的簇和相应的簇。在考虑对象和簇的关系时, 我们用到了点关于簇的归属度。为了方便描述点与簇的相似性, 我们定义了一个二值函数 $\chi(x_i, x_j)$, 如式 3-1 所示。

$$\chi(x) = \begin{cases} 1, & d(x_i, x_j) < T \\ 0, & d(x_i, x_j) \geq T \end{cases} \quad (3-1)$$

式中: T 是一个常量, $d(x_i, x_j)$ 表示数据对象 x_i 和 x_j 之间的距离。

定义 3-1 点关于簇的归属度。 点 x_i 关于簇 C_j 的归属度用簇 C_j 中与 x_i 的距离在常数 T 之内的点的个数来描述。用 $member(x_i, C_j)$ 来表示。其公式如式 3-2 所示。

$$member(x_i, C_j) = \sum_{e \in C_j} \chi(x_i, e) \quad (3-2)$$

定义 3-1 中定义的点关于簇的归属度用来表示点属于簇的可能性大小。通过点与簇内距离在一定范围之内点的个数来表示, 个数越多, 则点属于该簇的可能性越大。归属度的取值范围为 $[0, n]$, 其中 n 表示数据集中对象的个数。

图 3-2 是点关于簇的归属度的示意图。在图(a)中, 圆圈形状的对象表示同属

一个簇，用 C_c 来表示。正方形的对象同属另外一个簇，用 C_r 来表示。而中间的六边形是一个对象，用 P 来表示。在(b)中，虚线圆形的半径为常量 T 。我们来考虑 P 关于这两个簇 C_c 和 C_r 的归属度。从(b)中可以看到，簇 C_c 和点 P 的距离在 T 范围内的点共有 4 个，则 $member(P, C_c) = 4$ ，簇 C_r 和点 P 的距离在 T 范围内的点共有 3 个，即 $member(P, C_r) = 3$ 。

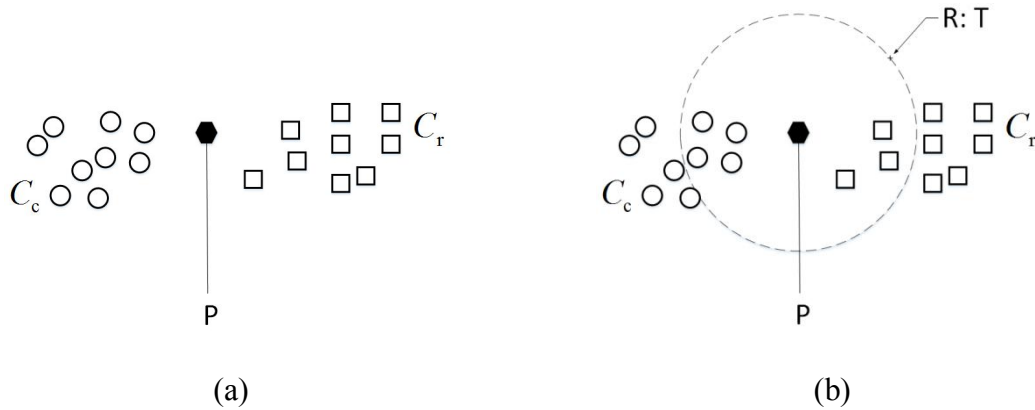


图 3-2 归属度示意图

在 CMSPC 的算法中，一个重要概念是近邻索引矩阵，如定义 3-2 所示。为了定义近邻索引矩阵，我们引入两个函数：近邻函数 $neigh(x_i, j)$ 和近邻索引函数 $neigh_ind(x_i, j)$ 。数据集 $X = \{x_1, x_2, \dots, x_n\}$ 中的每一个元素按照 $x_1, x_2, \dots, x_n$ 的顺序构成一个顺序列表，每一个元素都由它在这个列表中的索引 $1, 2, \dots, n$ 来表示。 $neigh(x_i, j)$ 用来表示元素 x_i 的第 j 个邻居。例如 $neigh(x_i, 1)$ 、 $neigh(x_i, 2)$ 分别表示距离 x_i 最近的元素和距离 x_i 次近的元素。 $neigh_ind(x_i, j)$ 用来表示元素 x_i 的第 j 个邻居在数据集 X 中的索引。

定义 3-2 近邻索引矩阵。对于数据集 X 中的元素 x_i 按照距离从小到大列出它的 n 个近邻索引，构成一个近邻索引序列。由数据集 X 中的所有元素的近邻索引序列构成的矩阵叫做近邻索引矩阵，如公式 3-3 所示。

$$neigh_ind = \begin{bmatrix} neigh_ind(x_1, 1) & neigh_ind(x_1, 2) & \cdots & neigh_ind(x_1, n) \\ neigh_ind(x_2, 1) & neigh_ind(x_2, 2) & \cdots & neigh_ind(x_2, n) \\ \vdots & \vdots & \ddots & \vdots \\ neigh_ind(x_n, 1) & neigh_ind(x_n, 2) & \cdots & neigh_ind(x_n, n) \end{bmatrix} \quad (3-3)$$

近邻索引矩阵的计算有两种方法，一种是先计算距离矩阵，再对距离矩阵的每一行进行排序，之后取索引就可以获得距离矩阵。但是这种算法的时间复杂度过高。使用 K-D Tree^[43] 算法来计算近邻索引矩阵可以将时间复杂度降低至 $O(n^2 \log_2 n)$ 。

3.2.2 基于点与簇内多点相似的聚类算法 CMSPC

本章提出的基于点与簇内多点的相似性的聚类算法将每个对象点看做是单个簇。通过考虑点关于簇的归属信息，对簇进行合并。当两个对象间的截断距离小于 T 时，我们认为这两个簇足够接近，这时考虑其中一点与另一点所在簇的关系。分两种情况考虑，对于两个数据对象 x_i 和 x_j ，如果 x_i 是单个簇的对象，则计算 x_i 关于 x_j 所在的簇 $C(x_j)$ 的归属度，若归属度 $member(x_i, C(x_j))$ 大于某个阈值，则簇 $C(x_i)$ 和簇 $C(x_j)$ 合并，即将 x_i 放入 $C(x_j)$ 。若 x_j 是单个簇对象，则要计算 x_j 关于 x_i 所在的簇 $C(x_i)$ 的归属度，若大于某个阈值，则将簇 $C(x_i)$ 和簇 $C(x_j)$ 合并，即将 x_j 放入 $C(x_i)$ 。若 x_i 和 x_j 都不是单个簇对象，则只要 x_i 关于 x_j 所在的簇 $C(x_j)$ 的归属度大于阈值或者 x_j 关于 x_i 所在的簇 $C(x_i)$ 的归属度大于阈值，就将簇 $C(x_i)$ 和簇 $C(x_j)$ 合并。聚类算法 CMSPC 的伪代码在算法 3-2 中展示。

算法 3-1 计算归属度，合并判断函数 merge_or_not

输入: $ind_A, ind_B, dist_matrix, neigh_ind, P$, 参数 T, M

输出: True 或 False

```

(1) if  $|C(B)| \leq M$  then
(2)   foreach  $e$  in  $C(B)$  do
(3)     if  $dist\_matrix[A, e] > T$  then
(4)       return False
(5)   return True
(6) else
(7)    $c \leftarrow 0$ 
(8)   foreach  $e$  in  $C(B)$  do
(9)     if  $c < M$  then
(10)      if  $dist\_matrix[A, e] \leq T$  then
(11)         $e \leftarrow e + 1$ 
(12)      else
(13)        break
(14)   if  $c \geq M$  then
(15)     return True
(16)   else
(17)     return False

```

在合并两个簇时是这样考量归属度的，当考虑的簇中的样本总数小于或等于

算法的输入参数 M 时, 则归属度须不小于簇中的样本总数, 才能进行簇合并; 若簇中的样本总数大于算法的输入参数 M , 则归属度须大于 M , 才能进行簇合并。截断距离 T 和归属度的阈值 M 是作为参数输入的。截断距离的建议值是使得数据集中所有对象在 M 之内的平均邻居个数占数据集的样本总量的 1% 到 2% 之间。计算归属度并判断是否合并 A 所在的簇 $C(A)$ 和 $C(B)$ 的算法伪代码在算法 3-1 中展示。

算法 3-2 聚类算法 CMSPC

输入: 数据集 X , 参数 T , 参数 M

输出: 类标列表 P

```

(1) 计算距离矩阵  $dist\_matrix$ , 得到近邻索引矩阵  $neigh\_ind$ 。初始化类标列表  $P$ 
(2) for  $i=1$  to  $n$  do
(3)   for  $j=1$  to  $n$  do
(4)     if  $j < i$  then
(5)       if  $dist\_matrix[i, j] \leq T$  then
(6)         if  $|C(i)| = |C(j)| = 1$  then
(7)            $p_i$  和  $p_j$  赋予同样的类标记
(8)         elif  $|C(i)| = 1$  and  $|C(j)| > 1$  then
(9)           if  $merge\_or\_not(i, j)$  then
(10)             $p_j \leftarrow p_i$ 
(11)         elif  $|C(i)| > 1$  and  $|C(j)| = 1$  then
(12)           if  $merge\_or\_not(j, i)$  then
(13)             $p_i \leftarrow p_j$ 
(14)         elif  $|C(i)| > 1$  and  $|C(j)| > 1$  then
(15)           if  $merge\_or\_not(i, j)$  or  $merge\_or\_not(j, i)$  then
(16)             foreach  $e$  in  $C(i)$  do
(17)                $p_e \leftarrow p_i$ 
(18)         else
(19)           break
(19) 将  $P$  列表中未赋值的每个元素都标为不同的标记, 意为异常点
(20) return  $P$ 

```

算法的输入是数据集 X 、截断距离参数 T 、归属度参数 M 。 X 的样本总量为 n , 表示为 $n \times m$ 的矩阵, m 是数据的维数。算法的输出为类标列表 P , 长度为 n , 其中的元素为字符。它表示数据集 X 中的元素的类标信息, 和 X 中的元素是相对应的, p_i 就表示 x_i 的类标。初始化时对于其中元素初始化为 -1, 表示该

元素为单个簇元素。所谓单个簇元素就是这个对象所在簇中只有它本身。算法首先计算了数据集的欧式距离矩阵,并获得近邻索引矩阵。这样,在算法中调用 x_i 和 x_j 之间的距离时只需要调用 $dist_matrix[i,j]$,而不需要每次都计算距离。通过近邻索引矩阵 $neigh_ind$,二层循环中,对于当前对象 x_i ,我们就从最近的邻居 $neigh_ind[i,1]$ 开始按照距离顺序进行考虑,当遇到的对象和 x_i 之间的距离大于截断距离 M 时,我们就可以终止二层循环,因为之后的对象和 M 之间的距离也必定大于 M 。这样大大减少了循环次数。算法 3-1 中 $C(i)$ 是临时簇,是对象 x_i 所在簇中的对象在数据集中索引的集合。

3.2.3 复杂度分析

欧式距离计算的时间复杂度是 $O(n^2)$,使用 K-D Tree^[43]算法来计算近邻索引矩阵的时间复杂度为 $O(n^2 \log_2 n)$,算法主体部分时间复杂度最坏情况是公式 3-4,其中 n 是数据集中对象的总数, k 为簇的个数, M 为算法的输入参数,这个值一般远小于 n , $|C_i|$ 表示第 i 个簇中对象的个数,一般也小于 n 。所以算法总的时间复杂度则为公式 3-5,化简之后为 $O(n^2 \log_2 n)$ 。

$$O(M * \max_{i=1}^k (|C_i|) * n^2) \quad (3-4)$$

$$O(M * \max_{i=1}^k (|C_i|) * n^2 + (\log_2 n + 1)n^2) \quad (3-5)$$

3.3 实验及结果分析

在这一节,我们设计了多个实验对 CMSPC 聚类算法的有效性和聚类质量进行评估。首先使用 4 个具有不同特点的人工合成的数据集来验证 CMSPC 的有效性。为了验证聚类算法 CMSPC 的聚类效果,我们在一些基准数据上和 K-Means、DBSCAN、层次凝聚的两种算法、AP 算法、谱聚类算法的聚类结果进行了对比。

3.3.1 实验环境及对比算法参数设置

(1) 实验环境

表 3-1 实验环境

处理器	Pentium(R) Dual-Core CPU E6600 @ 3.06GHz × 2
内存	5.8 GiB
操作系统	Ubuntu 15.10
编程语言	Python3.4
所有主要 python 库	Numpy、Sklearn

本实验的硬件和软件环境如表 3-1 所示。

(2) 对比算法及参数设置

在实验 2 中，我们在多个人工合成数据集和真实数据集上对 CMSPC 的聚类结果和其他聚类算法的聚类结果进行了对比。对比算法有 K-Means++、DBSCAN、WLC、ALC、AP 算法、谱聚类（Spectral）。对这些算法的参数设置描述如下。

在 K-Means++ 算法中，选择 k 个距离尽可能远的点作为初始聚类中心，能克服 K-Means 算法对初始中心点敏感的问题。我们输入正确的类个数作为参数，选取 10 组中心点，最终取使得目标函数最优值即 SSE（Sum of Squares Error）最小的初始中心点作为中心点进行聚类。DBSCAN 算法中有两个参数 ε 和 $MinPts$ ，实验中我们使用网格搜索，取聚类结果 ARI 值最高的结果作为最终结果。AP(Affinity Propagation) 算法中，我们取阻尼因子参数为 0.9，这样虽然增大了迭代次数，却能保证找到最优结果。偏好值参数我们取相似度的中值，最大迭代次数设为 1000。谱聚类（Spectral Clustering）中，输入参数为真实类个数。Ward linkage 凝聚层次聚类(WLC)中，输入真实类个数作为参数。Average linkage 凝聚层次聚类（ALC）中，输入真实类个数作为参数。

3.3.2 实验所用数据集

为了验证聚类算法 CMSPC 的有效性、聚类质量，我们在多个数据集上进行了实验。数据集包括人工合成数据集和基准数据集。

(1) 人工合成数据集

表 3-2 合成数据集

数据集名	样本总数	类个数	特点
DS-1	1500	3	球形
DS-2	1500	3	非球形（条状）
DS-3	1500	3	簇内密度不同
DS-4	610	3	簇大小不均衡
DS-5	1173	2	封闭环形

人工合成数据集是用了 Scikit-learn^[44]机器学习库中的函数生成的数据集。数据集的名称、样本总数、类个数、特点展示在表 3-2 中。

(2) 基准数据集

实验中使用的数据集主要来自 UCI 机器学习数据库^[45]、UEF 数据集^[46]、R 语言中的机器学习包^[47]。数据集的名称、属性个数、样本总量、类个数、来源等信息展示在表 3-3 中。我们在下文中简要描述了这些数据集。

表 3-3 数据集列表

数据集名	属性个数	样本总数	类个数	来源
Glass	9	214	7	UCI
Zoo	16	101	7	UCI
Newthyroid	5	215	3	UCI
Cleveland	13	197	5	UCI
Aggregation	2	788	7	UEF
Pathbased	2	300	3	UEF
Spiral	2	312	3	UEF
Jain	2	373	2	UEF
T4.8k	2	8000	6	UEF
Rface	2	600	6	R Package
Iris	4	150	3	UCI

(1) **Glass**: 玻璃数据集。犯罪现场留下的玻璃可以被用作物证, 在刑事案件的调查中, 正确识别出玻璃的类型对于案件的侦破很重要。该数据集共有 9 个属性, 用来描述玻璃中化学元素的含量。类别个数为 7, 用来表示不同类型的玻璃。

(2) **Zoo**: 动物园数据集。16 个属性包括 Hair、Features、Eggs、Milk、Airborne 等用来表示动物特征的属性。这些属性几乎都是二元属性, 除了 Legs 属性为个数为 6 的标称属性。该数据集的类别信息是动物所属类别, 一共有 7 个, 包含有哺乳动物、鸟类、爬行类、两栖、节肢等。

(3) **Newthyroid**: 甲状腺疾病数据集。包含五个属性。聚类的类别为患者当前状态, 包括正常、患有甲状腺功能亢进症、甲状腺功能减退三种。

(4) **Cleveland**: 心脏病数据集。该数据集是从包含有 76 个属性的数据中选出来 13 个有用的属性。聚类类别信息是患者当前状态, 1、2、3、4 表示该患者患有心脏病, 0 表示正常。

(5) **Aggregation**: 该数据集是 CFDP^[32]论文中用到的数据集, 来自于^[48]。该数据集是包含有椭圆和月牙形状的 2 维数据集, 该数据集有连接不紧密的两个簇。对于基于密度的 DBSCAN 聚类算法来说, 很难将这两个簇分开。

(6) **Pathbased**: 二维图形数据集, 来自于^[49]。

(7) **Spiral**: 该数据集是 CFDP^[32]论文中使用的数据集, 来自于^[49]。该数据集是 2 维的, 由三条螺旋线构成。对于基于划分的聚类算法来说, 很难进行聚类。

(8) **Jain**: 2 维图形数据集, 来自于^[50]。

(9) T4.8k: 包含有 6 个簇的 2 维图形数据集, 含有噪声, 来自于^[51], 该数据集不含真实类标。

(10) Rface: 来自于 R 语言包 fpc, 是一个人脸图形数据。

(11) Iris: 鸢尾花数据集。该数据集包括三种不同类型的鸢尾花, 一共 150 个样本。3 种花包括山鸢尾、杂色鸢尾、维吉尼亚鸢尾。山鸢尾比较容易和另外两种区分, 而另外两种线性不可分。4 个属性分别是花萼长度、花萼宽度、花瓣长度、花瓣宽度。花的类型作为聚类类别。

数据中不同属性的单位不同, 为了消除量纲, 实验中的所有数据都经过 min-max 标准化处理, 数据经过标准化处理之后, 都处在[0,1]区间之内, 标准化处理的公式如 3-6 所示。这些数据集中对象的属性有两种类型: 数值和标称。为了方便计算对象之间的相似性, 标称属性需处理成数值类型。

$$X_{sca} = \frac{X - \min(X, axis = 0)}{\max(X, axis = 0) - \min(X, axis = 0)} \quad (3-6)$$

3.3.3 CMSPC 聚类算法实验

实验部分一共包括 3 个实验, CMSPC 算法的聚类有效性实验、CMSPC 算法的聚类效果对比实验、CMSPC 算法的时间性能实验。

实验 1: CMSPC 算法的聚类有效性实验

有效性实验中, 我们使用了 5 个人工合成的数据集和一个基准数据集, 这些不同的 2 维数据集是为了验证 CMSPC 算法有效性的多个方面, 对算法的聚类有效性进行了实验。所用的数据集说明如表 3-2 和表 3-3 所示。

(1) 球形数据集

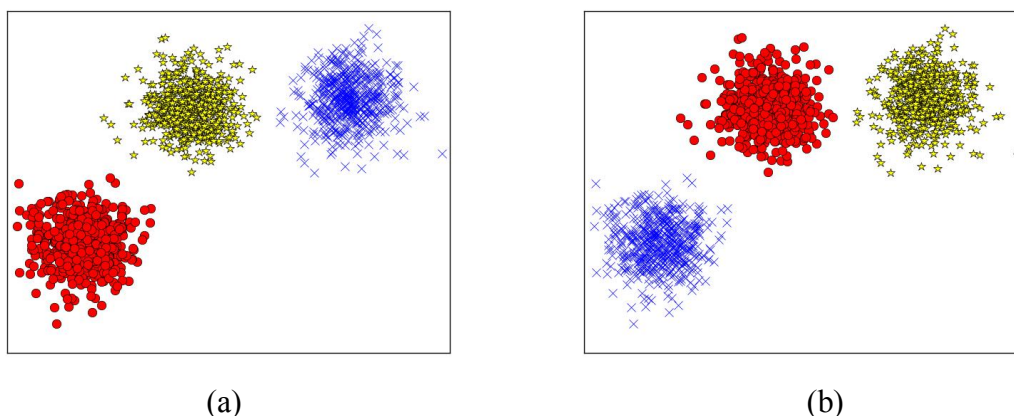


图 3-3 球形数据聚类效果展示 ((a)为真实结构, (b)为 CMSPC 聚类结果)

图 3-3 是对合成数据集 DS-1 的聚类实验图片, 从图中可以看到算法把这 3

个簇区分开了, 只有 3 个距离簇中心较远的点被当作异常点。可见, 对于不相交的球状簇数据集, CMSPC 聚类算法可以正确聚类。

(2) 非球形数据集

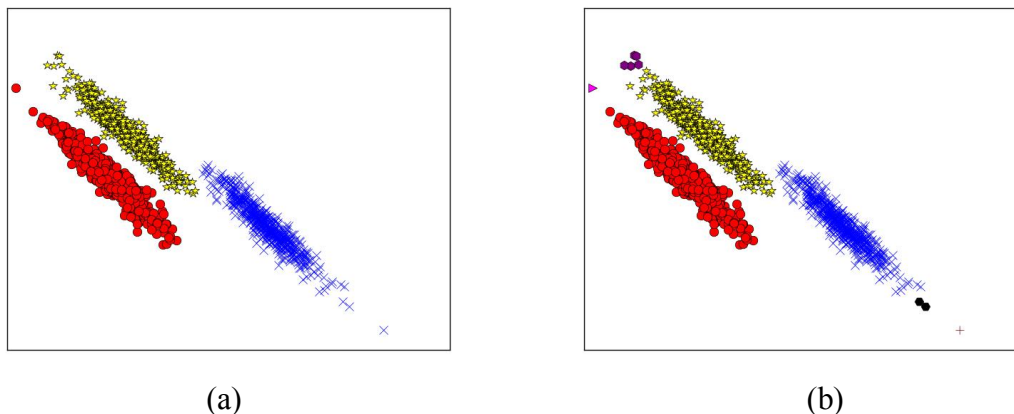


图 3-4 非球形数据聚类效果展示 ((a)为真实结构, (b)为 CMSPC 聚类结果)

图 3-4 是对非球形数据的聚类实验展示, 从图中可以看到大体簇结构完全正确, 距簇较远的几个点被当成了异常点来处理。聚类结果说明 CMSPC 算法对于这种非球形的条状数据可以很好得进行聚类。

(3) 簇密度不同的数据集

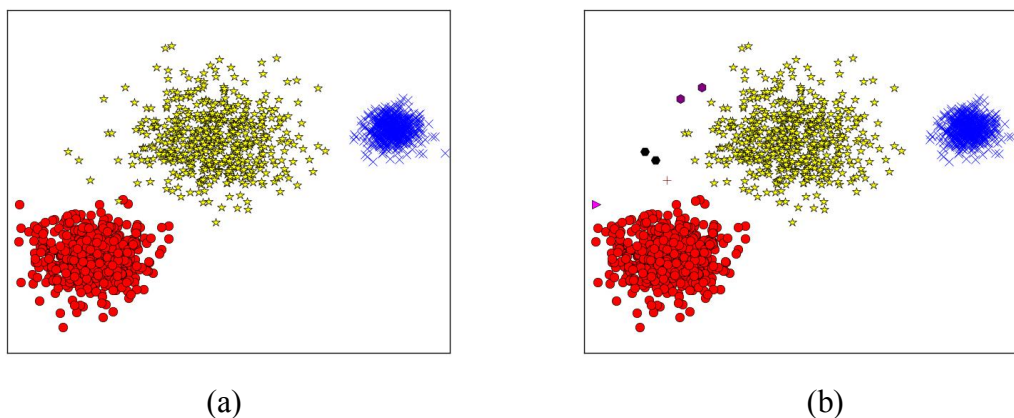


图 3-5 不同簇内密度数据集聚类效果展示 ((a)为真实结构, (b)为 CMSPC 聚类结果)

图 3-5 是 DS-3 数据集的实验图片展示。图 3-4(a)中五角星形状的数据簇内方差为 0.5, 圆圈表示的数据其簇内方差为 1.0, 而十字形状的簇为 1.5。图中(b)是 CMSPC 算法聚类结果图片展示, 只有极少数距离簇中心较远的数据点被错分了。该实验说明对于簇内密度不相同的簇, CMSPC 聚类算法能将这种簇正确识别出来。

(4) 簇大小不均衡数据集

图 3-6 是 DS-4 数据集的实验图片展示。图中(a)是真实的簇结构展示, 这三

个簇的样本数量从少到多依次为 10、100、500，这是一个簇大小不平衡的数据集。图中(b)是聚类算法的聚类结果展示，从图中可以看到，只有两个很稀疏的点被当作了异常点。实验表明 CMSPC 对于簇大小不平衡的数据集能正确进行聚类。

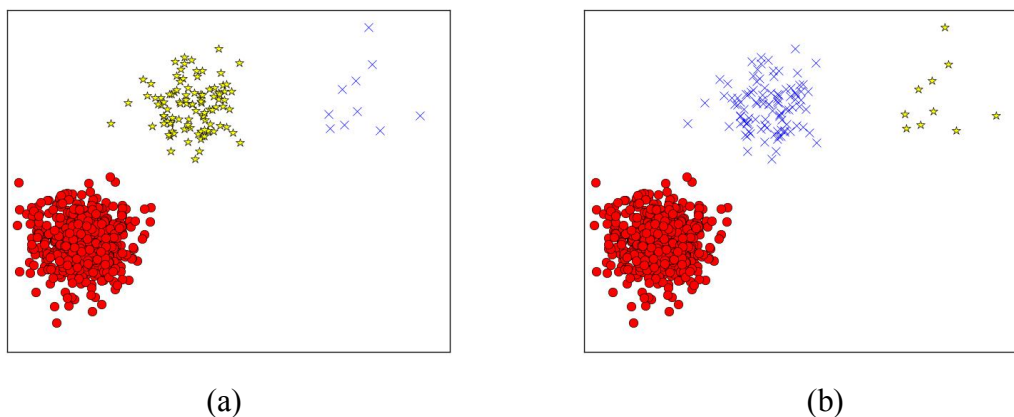


图 3-6 簇大小不平衡数据集聚类效果展示 ((a)为真实结构, (b)为 CMSPC 聚类结果)

(5) 多种形状数据集

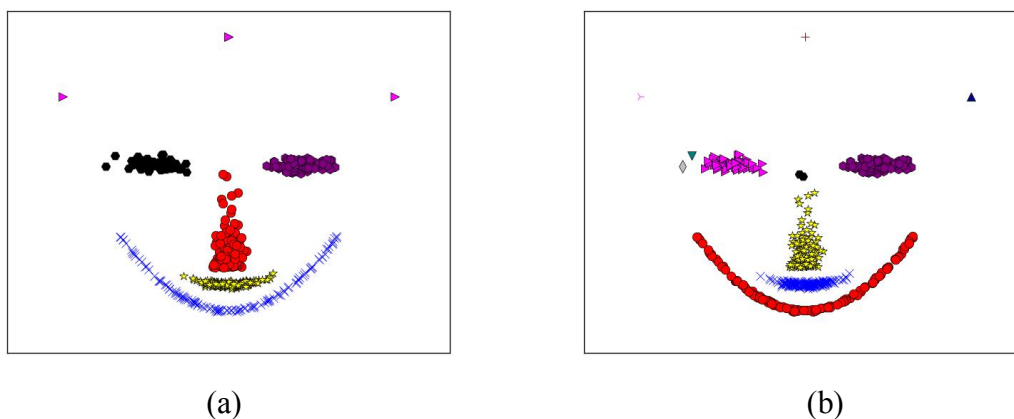


图 3-7 人脸图形数据的聚类效果展示 ((a)为真实结构, (b)为 CMSPC 聚类结果)

图 3-7 是 RFace 数据集的实验图片展示。RFace 是一个人脸结构图形。在这个数据集中，人脸的 6 个主要部分由不同形状的簇组成。从(b)可以看到，鼻子上边有两个点被错聚为另外一个类了，因为这两个点距鼻子主要部分太远。左边眼睛的两个点也分别被当作两个异常点了。额头部分原数据是把距离很远的 3 个点当作一类了。但是这样并不合理，本文认为这三个是 3 个异常点。算法也把这三个点识别为异常点了。实验说明 CMSPC 算法对于不同簇形状组合的数据集大体上能进行合适的聚类。

(6) 含有噪声的数据集

数据集 T4.8k 被许多研究者作为基准数据集来进行聚类效果测试，该数据集

不含类标。图 3-8 中 6 个粗条块为 6 个簇，周围是一些密度较稀疏的噪声点。可以看到，CMSPC 算法可以将这 6 个簇完全识别出来。实验说明 CMSPC 对于含有噪声点的簇结构具有很好的识别能力。

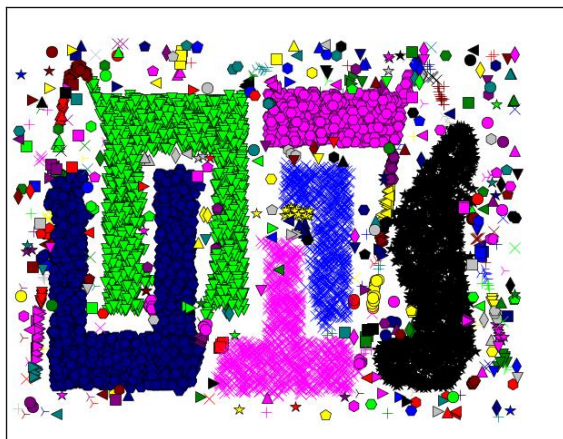


图 3-8 含有噪声的数据集 T4.8k 聚类效果展示

实验 2: CMSPC 算法的聚类效果对比实验

在本小节,我们将提出的算法 CMSPC 的聚类结果 ARI 和另外几个聚类算法在多个数据集上的结果进行了对比。这几个对比算法实验中的参数设置在 3.3.1 小结有叙述。聚类算法的对比实验结果在表 3-4 中列出。

表 3-4 对比实验聚类结果 ARI

数据集	K-Means++	DBSCAN	WLC	ALC	AP	Spectral	CMSPC
Aggregation	0.7151	0.9886	0.7218	0.8026	0.4136	0.5974	0.9891
Cleveland	0.1567	0.1177	0.1509	0.0314	0.0426	0.0757	0.2924
Glass	0.2288	0.2682	0.1960	0.0204	0.1808	0.1225	0.2820
Iris	0.7163	0.6246	0.7302	0.5596	0.4310	0.7184	0.6902
Jain	0.5767	0.9758	0.5607	0.0788	0.1043	0.7238	0.9758
Newthyroid	0.6283	0.5582	0.6283	0.1114	0.1291	0.6573	0.8363
Spiral	-0.0058	1.0000	-0.0052	0.5577	0.1334	0.2873	1.0000
DS-5	-0.0006	1.0000	-0.0004	1.0000	0.0585	1.0000	1.0000
Zoo	0.6936	0.9353	0.8318	0.4495	0.8812	0.5681	0.9416

从表 3-4 中可以看到,在表中列出的 10 个数据集上,和 K-Means++、DBSCAN、WLC、ALC、AP 算法、谱聚类相比,在其中 9 个数据集上,本文提出的算法 CMSPC 的 ARI 得分值都不低于另外五种算法。其中在数据集 DS-5 和 Spiral 上, CMSPC 的 ARI 得分值为 1,也就是说在这两个数据集上, CMSPC 能

完全聚类出正确的类结构。Spiral 是三条螺旋线，而 DS-5 是一个葫芦形的紧凑簇外部包含了一层稀疏的背景圈，两个簇的密度不同，但是簇内密度均匀，而且这两个数据集的簇结构都是非球形的，没有异常点和噪声点干扰，CMSPC 对这两个数据完全聚类正确。DBSCAN 算法在这两个数据集上也达到了同样好的聚类效果，但是在其他数据集上，DBSCAN 的聚类效果就不如 CMSPC 了。在 DS-5 数据集上，ALC 的 ARI 也达到了 1，而 WLC 的 ARI 值却为负数。这个很好理解，因为 average-linkage 是按照平均距离来作为簇间距离，数据集是本簇之内密度均匀，所以每次合并都是合并真实簇中的两个子集。而 WLC 使用的连接函数是最小化合并后的簇内方差。DS-5 和 Spiral 这种环状簇结构的数据簇内方差很大，尤其是外层簇。对于这种环状簇结构的数据集，ward 连接方法很难聚类正确。在 Iris 数据集上，CMSPC 算法没能比 K-Means++、Spectral 和 Ward 算法的聚类结果好。原因应该是 Iris 数据集的其中两个类非线性可分，这两个线性不可分的类边界是重叠的。为了了解 Iris 数据集的空间分布，我们将该数据集使用 PCA 降维到 2 维空间。图 3-9 中(a)是 Iris 数据集降维至 2 维空间后其真实类标的展示图片，可以看到蓝色星状的样本和另外两类是分离的，五角星标志的样本和圆圈标志的样本边界是重叠的。图 3-9 (b)是使用 CMSPC 聚类的效果图片展示，可以看到，对于线性可分的一类样本，CMSPC 基本可以分离出来，有少许距离簇中心较远的点，CMSPC 当成了异常点。而在另外线性不可分的两个类上，CMSPC 很难将其边界区分开来，但还是将这两类的大体结构识别了出来。这种数据集本身簇的边界很模糊，难以区分。

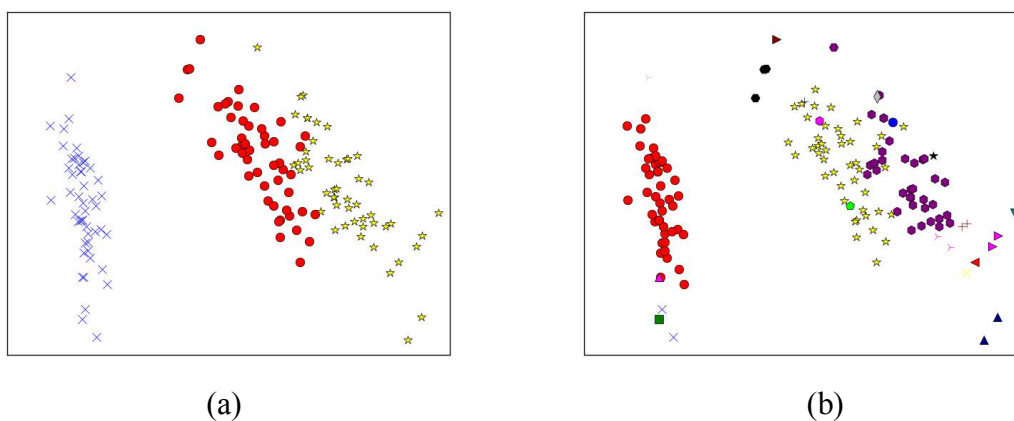


图 3-9 IRIS 数据在二维平面上的聚类效果展示 ((a)为真实结构, (b)为 CMSPC 聚类结果)

从实验 2 对比实验结果来看，和 K-Means++、DBSCAN、Ward、average-link、AP 算法、谱聚类相比，CMSPC 能在多数数据集上取得较这些算法更好的聚类结果。适用于簇间线性可分，簇内密度变化不大的数据集。CMSPC 算法不受簇形状的影响，能发现任意形状的簇，同时能识别出异常点。对于簇规模不均衡的数

据集，CMSPC 也能进行聚类。

实验 3: CMSPC 算法时间性能实验

为了验证 CMSPC 算法的时间复杂度，以 DS-1 数据集的分布为准，合成了数据规模为 1000、2000、4000、8000、10000、16000 的数据集。在这些数据集上运行 CMSPC 算法，记录执行时间，画出了折线图，如图 3-10 所示。图中实心原点的连线为 CMSPC 的运行时间折线图，横坐标为数据的规模，纵坐标是运行时间，单位为秒（sec）。图中的实线为时间复杂度为 $O(n^2 \log_2(n+3))$ 时的曲线图。可以看到，CMSPC 的时间性能曲线和复杂度为 $O(n^2 \log_2(n+3))$ 的曲线图整体上增长速度是一致的。这说明 CMSPC 算法的时间复杂度为 $O(n^2 \log_2(n))$ 。

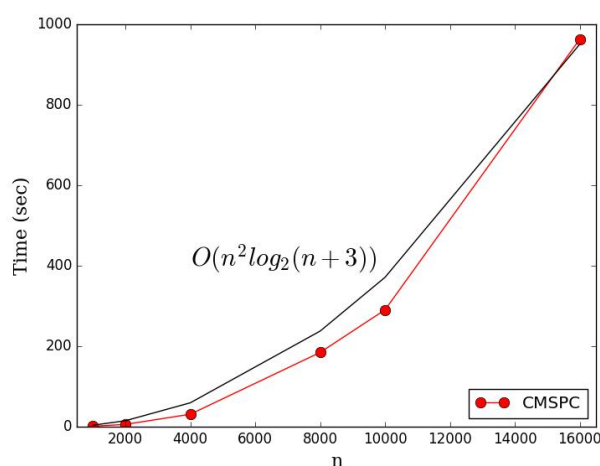


图 3-10 不同规模数据集下的聚类时间

3.4 本章小结

本章对现有的任意形状簇数据进行聚类的算法进行了分析，提出了一个基于点与簇内多点相似的聚类算法。该算法使用点关于簇的归属度信息，对满足条件的临时簇进行合并。该算法不需要输入类的个数作为参数，能自动识别出任意形状的簇，同时能识别出异常点。对具有不同特征的人工数据集进行了 CMSPC 的聚类有效性实验，实验结果说明 CMSPC 对于各种不同分布的数据集都能进行较好的聚类。我们在多个基准数据集上运行 CMSPC 算法，和 6 个其他聚类算法进行了对比，实验结果显示 CMSPC 在这些基准数据集上具有更好的聚类效果。对于 CMSPC 的时间复杂度进行了验证实验，实验结果说明 CMSPC 的算法时间复杂度为 $O(n^2 \log_2(n))$ 。

第四章 改进的聚类算法 CFDPm

CFDP 是一种简洁高效的的聚类算法，能识别出任意形状的簇，无需事先给定簇个数作为参数，仅有一个超参数 dc ，可用于高维数据。在众多的新型聚类算法中，CFDP 具有代表性。然而，因数据集的差异，并不是所有数据集的簇中心都能够通过 ρ 和 δ 来准确描述。选取不同的簇中心，聚类结果会完全不同。这降低了 CFDP 算法的稳定性和可用性。在本章中，我们首先介绍了 CFDP 算法，之后通过合并簇的方式来解决因簇中心的不准确选取带来的算法不稳定的问题，提出了基于 CFDP 的改进算法 CFDPm (CFDP modified)。在多个数据集上对新算法 CFDPm 的聚类有效性、聚类质量、时间性能等方面进行了实验。

4.1 CFDP 算法介绍

CFDP^[32]是一种新的快速聚类算法。该算法将局部密度最大的对象看作是簇中心，它认为簇中心就是那些比周围的对象具有更高的密度，且距离比其密度更大的对象较远的点。该算法定义了两个变量：局部密度 ρ 和距离更高密度点的距离 δ 。 ρ 和 δ 这两个变量都是通过对象之间的距离来定义。 ρ 的定义如公式 4-1 所示。 δ 的定义如公式 4-3 所示，特别地对局部密度最大的对象 δ 定义为距其最远的元素之间的距离。 ρ 和 δ 这两个变量明显较大的那些对象被看作是簇中心点。算法首先确定簇中心点，之后对于剩余的对象按照局部密度 ρ 从大到小进行排序，剩余点的类标被分配为比其密度大的距其最近的对象的类标。这样，整个数据集按照从簇中心到簇边界的顺序使用类标传递的方式进行聚类。

$$\rho_i = \sum_{j=1}^n \chi(d_{i,j} - d_c) \quad (4-1)$$

$$\chi(x) = \begin{cases} 1, & d(x_i, x_j) < T \\ 0, & d(x_i, x_j) \geq T \end{cases} \quad (4-2)$$

$$\delta_i = \begin{cases} \min_{j: \rho_j > \rho_i} (d_{i,j}), & \rho_i \neq \max_{q \in \{1, 2, \dots, n\}} (\rho_q) \\ \max_{q \in \{1, 2, \dots, n\}} (d_{i,j}), & \rho_i = \max_{q \in \{1, 2, \dots, n\}} (\rho_q) \end{cases} \quad (4-3)$$

CFDP 算法具有时间复杂度低，聚类效果好的特点。并且无需事先给定类的个数以及其他输入参数，算法就能自动识别出来簇中心。CFDP 在论文^[32]的附加

材料中给出的算法程序是一个交互式的程序。程序首先根据数据输入计算每个对象的 ρ 和 δ 。之后以 ρ 为横坐标、 δ 为纵坐标画出一个决策图，用户可以通过鼠标点击拉出一个矩形框来选择位于右上角的 ρ 和 δ 明显较大的点作为簇中心，之后程序自动对剩余节点进行分配，最终给出聚类结果。

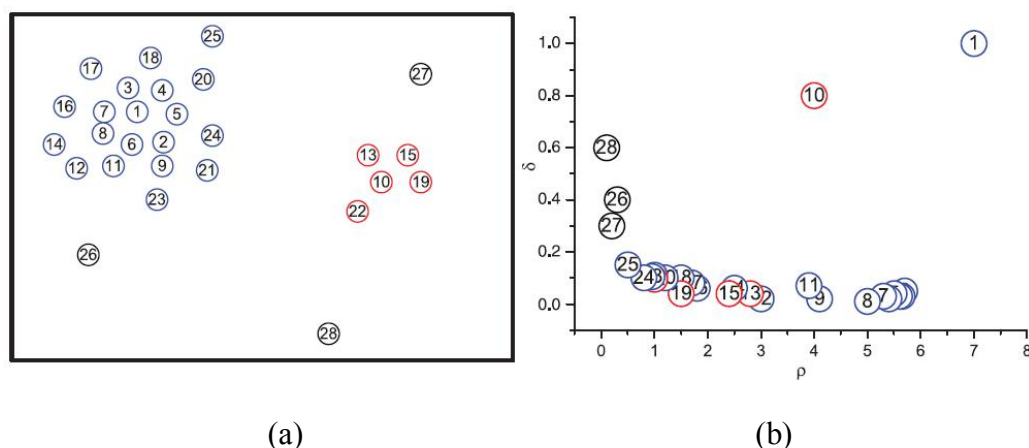


图 4.1 CFDP 算法示意图^[32]

图 4-1 是 CFDP 算法的一个示意图。图 4-1 (a) 中一共有 28 个点代表数据集中的 28 个对象。局部密度从 1 到 18 依次递减。图 4-1 (b) 中是这 28 个点的决策图，横坐标是局部密度 ρ ，纵坐标是 δ 。从图 4-1 (b) 中可以看到只有点 1 和 10 具有较大的 ρ 和 δ 。算法运行过程中，会提供给用户一个交互界面，用户通过鼠标划出一个矩形框来选取用户认为的簇中心点。对于图 4-1 中的数据集来说，比较容易来决定选取 1 和 10 两个点作为簇中心点。当 1 和 10 被选取后，剩余的点会被按照 ρ 从大到小的顺序，将类标标为距其最近的比其密度更大的点相同的类标。而 ρ 较小 δ 却很大的点会被认为是异常点。这样，图 4-1 中的点会被聚为两个簇，10,13,15,19,22 是一个簇，簇中心是 10。而 26、27、28 是异常点。剩余的点被聚为一个簇，簇中心是 1。

CFDP 算法对于分布特点类似于图 4-1(a) 中的数据集，聚类效果很好。然而，在 CFDP 算法中，簇中心点如何选取，其作者并没有给出来确定的描述。因为“ ρ 和 δ 明显较大”并没有一个定量标准，不同的用户可能会选择不同个数的点作为中心点，最终的聚类结果也会因选取中心点的个数不同而不同。因为数据集的分布不同，簇内的对象密度不同，所以也很难给出这样一个标准。

4.2 CFDPm 算法

本节将介绍本文提出的基于 CFDP 算法的改进算法 CFDPm，该算法是根据

簇内距离、簇间距离、聚类内部评价指标 DBI 对 CFDP 的聚类结果进行合并的一种算法。

4.2.1 CFDP 算法在多峰簇数据集上的聚类结果分析

有些数据集一个簇内存在多个距离较远的局部密度极大值点,这也会对中心点的选取带来困难,我们将这样的簇称作多峰簇,将含有多峰簇的数据集称作多峰簇数据集。当 CFDP 聚类算法被用于多峰簇数据集聚类时,用户选取的 ρ 和 δ 极大的点个数通常会比实际的类个数要多,这样,因选取的簇中心过多会造成聚类效果较差。我们在一个人工合成的数据集上进行了实验,图 4-2 中进行了展示。

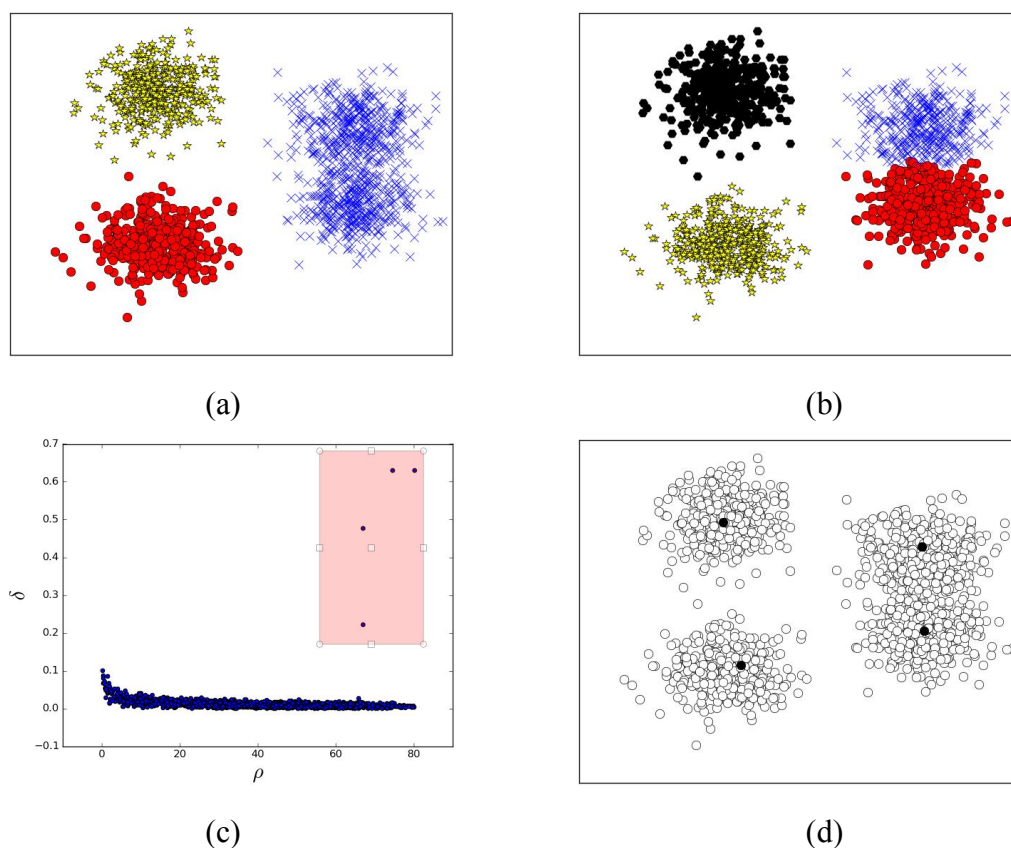


图 4-2 DS-6 数据集的 CFDP 算法聚类效果展示

((a)为真实簇结构;(b)为 CFDP 聚类结果;(c)为决策图;(d)是以(c)图中的簇中心选取的点在数据集中所对应的 4 个簇中心点的展示,其中黑色实心原点代表所选取的中心点。)

从(c)中可以看到,有 4 个点的 ρ 和 δ 值明显大于其余的点,于是选择这四个点作为簇中心点。(b)是按照(c)图中的簇中心选取后 CFDP 算法的聚类效果展示图,可以看到数据集被聚成了 4 个簇。和(a)图真实簇结构对比,可以看到右边的簇被错分为两个簇。(d)中右边的两个黑色实心圆点是两个较远的局部密度极大值,这两个点的 ρ 和 δ 都比较大,在决策图中很容易都被看作是簇中心点。最终导致数据集被错分为 4 个簇。而在真实簇结构中,这两个点属于同一个簇。

CFDP 遇到的问题是选取过多簇中心点造成的, 由于数据集分布的不同, 对于选取哪些点作为簇中心点, 很难给出一个定量的描述。本章给出的解决方法是对于 CFDP 聚类算法的聚类结果再进行一次合并, 对于距离较近的簇, 当满足一定的合并条件时予以合并。这样便能解决簇中心点选取过多所带来的麻烦。在考虑合并两个簇的时候, 我们将簇间距离、簇内距离, 以及簇结构的内部评价指标 DBI 度量结合起来进行操作。

4.2.2 CFDPm 算法思想

在对多峰簇数据集以及其上 CFDP 聚类结果分析之后, 我们发现多峰簇被分为的多个子簇的簇内密度相近, 且子簇间距离很小。对于 CFDP 的聚类结果, 我们通过簇内密度是否相近和簇间距离是否足够小来判断两个簇是否属于多峰簇的两个子簇。借助内部评价指标 DBI, 若两个簇的密度相近, 且簇间距离足够小, 并且合并它们之后能够降低整体的 DBI, 我们就认为这两个簇是多峰簇的两个子簇, 应该将它们合并。DBI 指标在本文第二章 2.4.2 节有所介绍。在 CFDP 算法中, 当不同的用户选择不同个数的点作为簇中心点时, 聚类结果不一样。而 CFDPm 算法加入了合并这一步骤, 对 CFDP 算法的聚类结果进行判断, 合并那些分布类似、距离较近、合并后能提升总体 DBI 指标的簇。

4.2.3 相关定义及概念

CFDPm 的合并步骤使用了簇内距离、簇间距离和评价指标 DBI。簇的密度可以用簇内平均距离来表示, 如定义 4-1 所示。簇间距离我们使用了最小距离。

定义 4-1 簇内平均距离。簇内平均距离是簇内所有对象到其最近邻居对象距离的平均值。簇 C_i 的簇内平均距离如公式 4-4 所示。

$$dist(C_i) = \frac{\sum_{x_j \in C_i} |x_j - neigh(x_j, 2)|}{|C_i|} \quad (4-4)$$

式中: $neigh(x_i, 2)$ 在第三章的 3.2.1 有提到, 它是一个近邻函数, $neigh(x_i, 2)$ 用来表示除去本身外距离 x_i 最近的对象。

簇间距离。簇间距离用最小距离来表示, 其公式在第二章中有定义, 如式 2-8 所示。本章提到的簇间距离特指最小距离, 也就是最小连接方法。

4.2.4 CFDPm 算法详细描述

本文提出的 CFDPm 算法是基于 CFDP 算法的, 在 CFDP 算法之后增加了一步合并过程。算法包含两个阶段, 第一个阶段是对数据集进行 CFDP 聚类, 第二个阶段是对 CFDP 的聚类结果进行合并。对于 CFDP 的聚类结果 C , 我们考虑 C

中任意的两个簇，若两个簇 C_i 、 C_j 具有相似的簇内平均距离，并且两个簇的最小距离和这两个簇内的平均距离也相近，那这两个簇 C_i 、 C_j 极有可能是多峰簇中的两个子簇，这两个簇被称为待合并簇。但是也有可能是比较接近的两个不同的簇。这个时候，我们使用 DBI 指标来进行判断，如果合并 C_i 和 C_j 能降低整体的 DBI 值（DBI 值越小，聚类结果越好），我们就认为这两个簇就是多峰簇中的两个子簇，应该予以合并。簇内平均距离接近到什么程度以及簇间距离小到什么程度时合并两个簇？为了量化这两个指标，我们在多个多峰簇数据集上进行了实验，最终确定了这样的标准：当两个簇 C_i 和 C_j 中平均距离较大者的平均距离小于 2 倍的较小者的平均距离时，我们认为这两个簇的平均距离相差不大，如公式 4-6 所示。当两个簇的最小距离小于 3 倍的平均距离较小者的平均距离时，我们认为这两个簇间距离足够小，如公式 4-5 所示。

$$d(C_i, C_j) < 3 \min(\text{dist}(C_i), \text{dist}(C_j)) \quad (4-5)$$

$$2 \min(\text{dist}(C_i), \text{dist}(C_j)) > \max(\text{dist}(C_i), \text{dist}(C_j)) \quad (4-6)$$

算法 4-1 CFDPm 聚类算法

输入: 数据集 X

输出: 划分 C

- (1) 使用 CFDP 对数据集 X 进行聚类，得到簇结构 C ，计算 DBI 得分 $\text{DBI}(C)$
 - (2) 初始化待合并列表 mergePairs 为空列表。
 - (3) **for** $i=1$ **to** k **do**
 - (4) **for** $j=1$ **to** k **do**
 - (5) **if** $j < i$ **then**
 - (6) **if** $d(C_i, C_j) < 3 \min(\text{dist}(C_i), \text{dist}(C_j))$ **then**
 - (7) **if** $2 \min(\text{dist}(C_i), \text{dist}(C_j)) > \max(\text{dist}(C_i), \text{dist}(C_j))$ **then**
 - (8) $C' \leftarrow C$
 - (9) 合并 C_i 、 C_j 在 C_{copy} 中对应的 C'_i 、 C'_j ，计算 $\text{DBI}(C_{\text{copy}})$
 - (10) **if** $\text{DBI}(C_{\text{copy}}) < \text{DBI}(C)$ **then**
 - (11) 将 p_i 和 p_j 组成的二元组 (p_i, p_j) 放入 mergePairs
 - (12) **foreach** (p_j, p_i) **in** mergePairs **do**
 - (13) 合并簇 C_i 、 C_j
 - (14) **return** C
-

CFDPm 算法的伪代码如算法 4-1 所示。算法的输入是数据集 X ，它是形状为 $n \times m$ 的矩阵。算法的输出是簇结构 C 。算法一共有两个阶段，CFDP 聚类和合并阶段。首先对于数据集 X 使用 CFDP 算法得到聚类结果的簇结构 C 。我们初始化一个合并列表，用来存放待合并的两个簇对应的类标记。接下来就是合并阶段了。对于 CFDP 的结果 C ，我们考虑任意两个簇 C_i 、 C_j ，若这两个簇的簇内平均距离的接近程度和簇间距离的值满足式 4-5 和式 4-6，则这两个簇被认为是待合并簇。之后尝试合并待合并簇，如果合并后整体的 DBI 降低了，这两个簇便可以合并，我们将这两个簇对应的类标对 (p_i, p_j) 放入合并列表 `mergePairs`。在所有簇对都被考虑了之后，我们就对 C 进行实际合并，从 `mergePairs` 列表中依次取出簇对，进行合并，最后返回 C 。

4.3 实验及结果分析

为了测试 CFDPm 的合并过程对 CFDP 算法能正确聚类的数据集是否也能保证正确聚类，我们使用 CFDP 算法的论文中的三个数据集进行了测试。为了验证 CFDP 算法的改进算法 CFDPm 的聚类效果，我们在多个基准数据集上进行了实验。为了验证 CFDPm 的稳定性，在这些基准数据集上设计并进行了实验。

4.3.1 实验数据集

实验部分一共使用了 DS-6、Aggregation、Spiral、Flame、Cuboids、Cassini、Shape、Heart、Saheart、Cleveland 10 个数据集。其中 Aggregation、Spiral、Cleveland 数据集在本文 3.3.2 节介绍。数据及都经过 0-1 标准化处理，以消除量纲。

(1) DS-6: 人工合成数据集，共包含三个簇，其中一个簇是由两个重叠的高斯分布混合而成。

(2) Flame: 是 CFDP 论文中用到的数据集，来自于^[52]。该数据集是火焰形状的数据集，包含有两个类。其特点是簇边界不明显。

(3) Cuboids: 来自于 R 语言包 `mlbench`^[53]机器学习基准库，该数据是一个三维数据集，外层是三个立方体，中间是一个小立方体。

(4) Cassini: 来自于 R 语言包 `mlbench`^[53]机器学习基准库，是一个二维数据集，包含三部分，内层是一个椭圆，外层两边是包围着椭圆的两个香蕉形状。

(5) Shape: 来自于 R 语言包 `mlbench`^[53]，是一个二维数据集，由四个分离的形状不同的图形组成。

(6) Heart: 该数据集是心脏病数据集，有 3 个属性，一共有 270 个样本，包含两类“患病”和“未患病”。

(7) **Saheart**: 南非西开普省心脏病高发区域男性的回顾性样本。一共 462 个样本, 包含冠心病阳性和阴性。包含有数值型属性和标称型的 9 个属性。对于标称型属性, 我们会将它转换成为数值型, 以方便在算法中计算距离。

4.3.2 CFDPm 算法实验

实验部分包括 CFDPm 算法的聚类有效性实验、多个基准数据集上的聚类效果实验、选取不同簇中心点时的聚类效果稳定性实验、时间性能实验 4 个实验。

实验 1: CFDPm 算法的聚类有效性实验

CFDPm 算法的有效性实验包含两个方面, 第一是通过验证在增加了合并步骤之后, 不会因合并不准确而影响 CFDP 算法正确分类结果。第二是要通过实验验证我们提出的合并思想是有效的, 可以将错分的簇合并起来。在这个实验中, 我们一共用到了 DS-6、Aggregation、Flame、Spiral、Shape 这 5 个数据集。两个算法的聚类结果的效果图对比如图 4-4 所示。

图 4-4 中, (a)、(b)、(c)、(d)、(e)、(f)、(g)、(h)、(i)、(j) 分别是 DS-6、Aggregation、Flame、Spiral、Shape 这 5 个数据集各自的 CFDP 和 CFDPm 算法的聚类效果图片展示, 左边是 CFDP 算法的聚类结果, 右边是 CFDPm 算法的聚类结果。如(a)、(b)分别是 CFDP 和 CFDPm 在 DS-6 数据集上的聚类效果图, 可以看到 CFDP 将右边的簇分为两个簇了, 而 CFDPm 算法聚类正确。CFDP 和 CFDPm 算法在 Flame 和 Spiral 这两个数据集上聚类结果是一样的, 如(e)、(f)、(g)、(h)所示, 在这两个数据集上, CFDPm 算法保持了 CFDP 算法的聚类结果, 并没有进行合并。(f) 图中两个簇密度相当, 簇间距离也很近, 界限不清晰, 之所以没将它们合并, 是因为 CFDPm 算法合并步骤中的簇间距离和 DBI 条件都没有被满足。又如(d)图所示, CFDPm 算法对于 Aggregation 数据集的聚类结果, 图中右边的两个相连的椭圆簇, 和 CFDP 算法对这两个椭圆点集的聚类结果一样, 也被正确得分成了两个不同的簇。这两个椭圆簇数据密度相当, 距离又足够近, CFDPm 算法却没有将它们合并, 原因是这两个簇还没有满足合并步骤的 DBI 指标。结合 CFDPm 算法在 Flame 和 Spiral 数据集上的聚类结果, 说明增加的合并过程并不影响 CFDP 聚类结果中正确的部分。再对比(c)、(d), 可以看到 CFDP 算法把同一簇的样本错分成了两个簇, 而 CFDPm 算法正确得分为了一个簇, 这是因为 CFDPm 算法在合并步骤将错分的两个子簇根据合并条件合并起来了。再对比(a)、(b)和(i)、(j), CFDPm 算法将 CFDP 算法错分为两个簇的簇正确合并了起来。这说明 CFDPm 算法中的合并步骤能将 CFDP 算法错分的簇合并起来。CFDPm 算法在解决 CFDP 算法在多峰簇数据集上失效问题时是有效的。

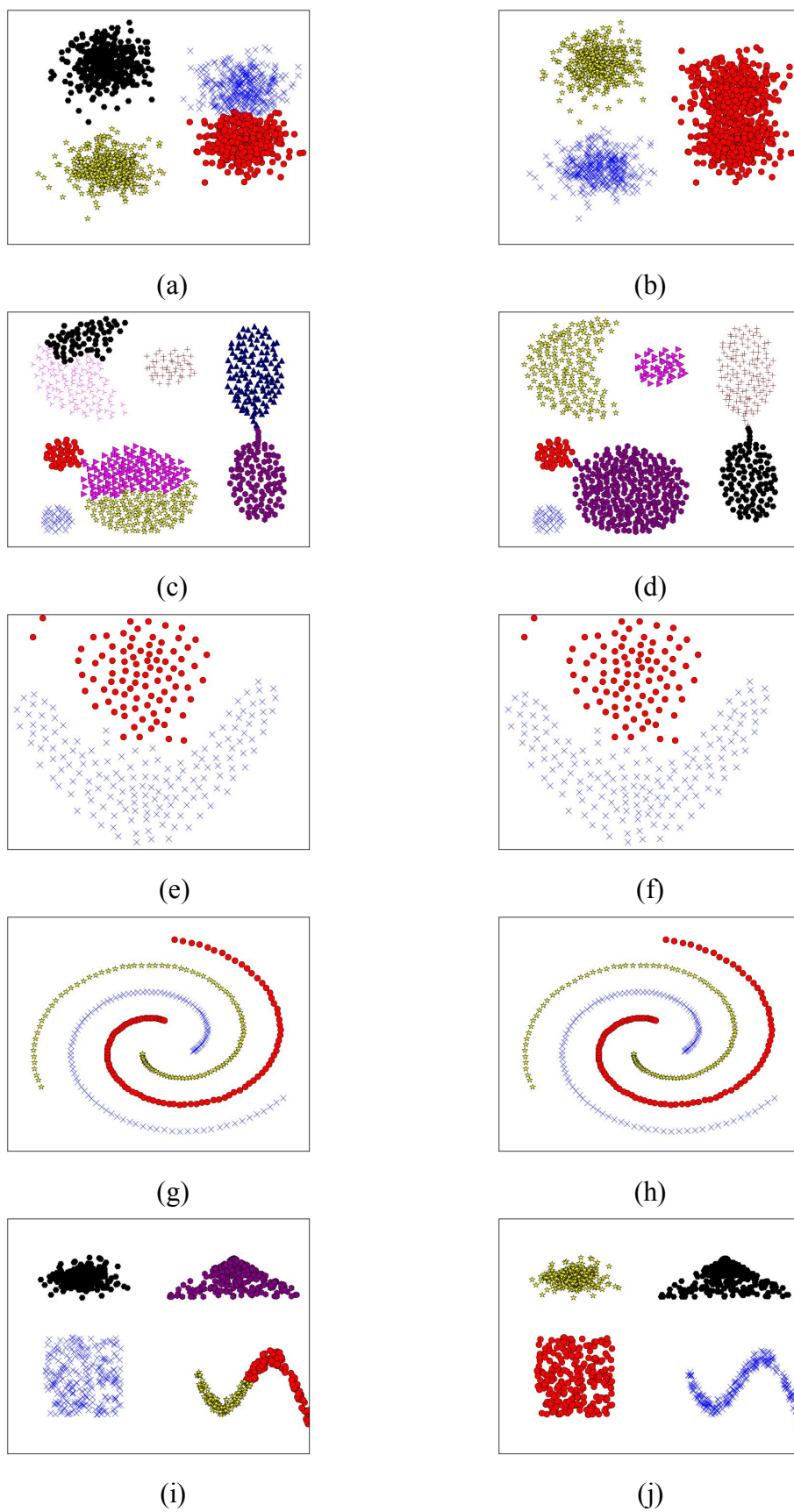


图 4-4 CFDP 和 CFDPm 聚类效果（左边为 CFDP 结果，右边为 CFDPm 结果）

实验 2: CFDPm 算法在多个基准数据集上的聚类效果实验

实验 1 中,我们通过在 5 个数据集上实验证明 CFDPm 算法在解决 CFDP 在多峰簇数据集上失效的问题是有有效的。为了进一步验证 CFDPm 聚类效果的好坏,我们在更多的基准数据集上进行了实验,这些数据集中有些都是来自真实世界。在实验中,对于每一个数据集,我们运行了 CFDP 算法和 CFDPm 算法,使用同样的超参数 dc ,选取同样的点作为簇中心点。超参数 dc 的设置原则是使得以 dc 为半径的邻域内,平均邻居个数达到样本数的 1%到 4%之间。在实验中,我们先使用 CFDP 的默认值 2%,如果不能令算法 CFDP 算法达到较好的聚类结果,我们会以 0.5%的步长增加或减少,以选取到合适的 dc 值,使得 CFDP 算法有效。在选取簇中心点上,我们尽量多的选取 ρ 和 δ 明显较大的点作为簇中心点。两个算法的聚类结果 ARI 对比如表 4-1 所示。

表 4-1 DFCP 和 DFCPm 算法的聚类结果对比

数据集	dc (参数)	CFDP (ARI)	CFDPm (ARI)
Cuboids	0.1022	0.5203	0.9073
Shape	0.0370	0.9128	1.0000
Heart	0.7028	0.0896	0.1895
Cleveland	0.5918	0.0757	0.2427
Saheart	0.2999	0.0215	0.0902
Cassini	0.0690	0.3289	0.7607
Aggregation	0.0677	0.7146	0.9956
Flame	0.0914	1.0000	1.0000
Spiral	0.0799	1.0000	1.0000

从表 4-1 中列出的数据可知,在这 9 个数据集上,CFDPm 算法的聚类结果 ARI 均不低于 CFDP。在 Cuboids、Heart、Cleveland、Cassini、Aggregation 数据集上,CFDPm 算法的 ARI 远高于 CFDP 算法。实验表明 CFDPm 算法具有比 CFDP 算法更好的聚类效果。

实验 3: CFDPm 算法在选取不同簇中心点时的聚类效果稳定性实验

为了对比 CFDP 算法和 CFDPm 算法在选取不同个数的点作为簇中心点时算法的稳定性,我们又在实验 2 中用到的数据集上进行了聚类的稳定性实验。这个实验中我们多次选择不同个数的点作为簇中心点,选取方式是:以真实类个数为下限,以所有 ρ 和 δ 明显较大簇的个数为上限,选择可能成为簇中心点的点作为簇中心点。在这些数据集上,使用相同的超参数 dc 分别执行 CFDP 和 CFDPm

算法，并记录聚类结果 ARI，最后将聚类 ARI 得分值以带误差棒的折线图的形式展示在图 4-5 中。

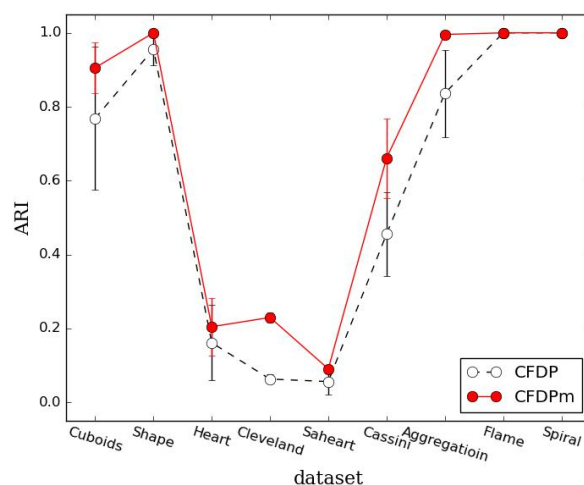


图 4-5 CFDPm 算法和 CFDP 算法选取不同个数簇中心点时的折线图

图 4-5 展示了两条 errorbar 折线图。空心圆的折线图是 CFDP 算法的结果展示，实心圆的折线图是 CFDPm 算法的结果展示。其中横坐标是数据集名称，纵坐标是 ARI 得分值。图中圆点处对应的纵坐标值为多次选取不同个数的簇中心点后聚类结果的平均值，errorbar 中的 error 使用了多次聚类结果的标准差。由图可知，在这 9 个数据集上，CFDPm 算法的平均 ARI 得分值均大于 CFDP 算法的平均 ARI。从 errorbar 上来看，总体上 CFDPm 算法稳定性相对也较好。为了分析和说明 CFDPm 稳定性的原因，我们将 Aggregation 数据集 3 次不同簇中心点选取的决策图，以及两种算法的聚类结果图画了出来，展示在图 4-6 中。

图 4-6 中(j)是 Aggregation 数据集的真实簇结构图。图中第一行是选取 7 个点为簇中心点时的图片展示，(a)、(b)、(c)分别为 CFDP 算法聚类结果图、CFDPm 算法聚类结果图、簇中心选取图。第二行和第三行分别是选择 8 个、9 个簇中心点时对应的图片展示。图中(k)是按照(i)选取的 9 个簇中心点在数据集中所对应的点的展示图，用黑色实心圆点来表示所对应的中心点。(j)是 Aggregation 数据集的真实簇结构 2 维展示图。从图中可以看到，(b)、(e)、(h)是一样的，无论选择 7、8、9 个簇中心点，CFDPm 算法都能将错分的子簇正确合并起来，聚类结果和真实结构基本相符，只有位于右边的两个椭圆的边界处有两个点被错分了。而对于 CFDP 算法来说，只有当选择的簇中心点和真实簇结构的簇中心点个数相同时聚类效果才比较好，如(a)所示。图中(b)、(e)、(h)和(a)是一样的。

实验 4: CFDPm 算法时间性能实验

为了分析 CFDPm 算法的时间性能，我们按照 DS-6 的分布生成了不同规模

的样本。我们生成的样本的样本总量分别为 1000、2000、4000、6000、8000、10000，在这些数据集上分别执行 CFDP 算法和 CFDPm 算法。两种算法在不同规模的数据集上执行的时间性能如图 4-7 所示。

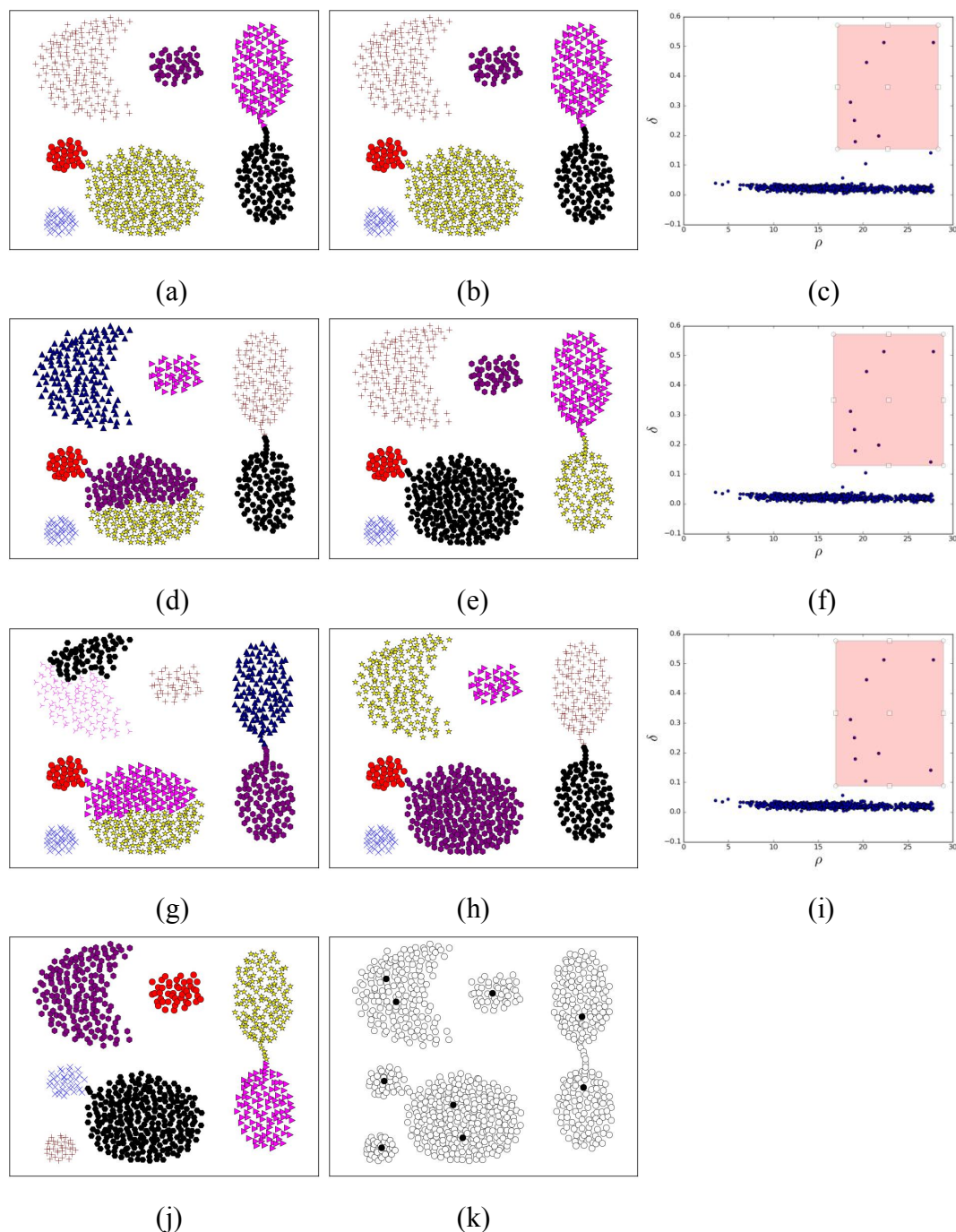


图 4-6 CFDPm 算法在 Aggregation 数据集上的选取不同簇中心点

图 4-7 中不带标记的黑色实线是时间复杂度为 $O(n^2)$ 的曲线图。从图中可以看到，CFDP 和 CFDPm 算法执行时间的曲线基本重合，且接近 $O(n^2)$ 级别的时间复杂度曲线。两个算法的执行时间曲线重合的原因可应该是 CFDPm 算法增加

的合并过程时间复杂度很小，而时间复杂度最大的部分主要是计算距离的工程。算法计算距离的时间复杂度为 $O(n^2)$ 。

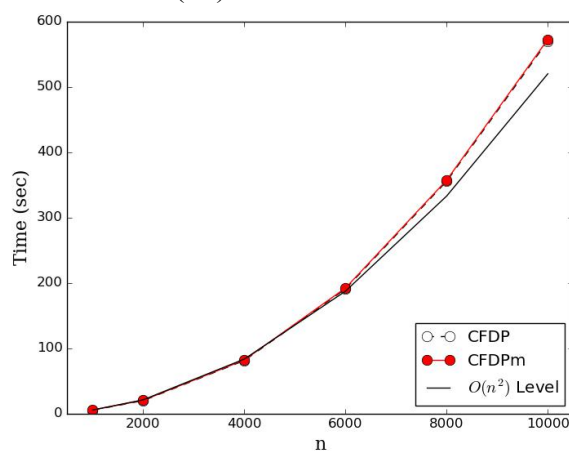


图 4-7 CFDP 和 CFDPm 在不同规模数据集上的执行时间

4.4 本章小结

本章对新型算法中具有代表性的 CFDP 算法进行了研究，针对其簇中心点选取所带来的聚类结果不稳定性的特点进行了分析。提出使用簇内距离和簇间距离以及聚类评价指标对 CFDP 算法的聚类结果进行评估、合并。基于这个思想，我们提出了基于 CFDP 算法的改进算法 CFDPm。在 10 个数据集上，我们设计了 4 个实验分别来对算法聚类的有效性，聚类结果的准确性，选取不同簇中心点时聚类结果的稳定性方面以及算法的时间性能进行了实验。实验结果说明 CFDPm 算法具有比 CFDP 算法更好的聚类结果，且具有更好的稳定性。而合并过程不增加算法的时间复杂度。

第五章 总结与展望

5.1 本文工作总结

聚类是数据挖掘中的一个基础工具, 尽管聚类的研究经历了半个世纪之久, 但是数据的多样性以及对聚类的新要求不断出现, 聚类算法还在层出不穷。尤其在进入二十一世纪之后, 随着互联网的发展以及计算机技术广泛应用, 各行各业来自数据库、数据仓库、网络的不同类型的数据爆炸式增长。空间数据来自于空间数据库、生物医学数据、农业领域, 空间数据的特点是包含有大量的任意形状的簇。这对现有的聚类算法提出了挑战, 本文主要研究挖掘任意形状簇的聚类算法, 主要提出了两个聚类算法 CMSPC 和 CFDPm。

(1) CMSPC 是一种基于点与簇内多点相似的聚类算法。该算法引入点关于簇的归属度, 对于满足截断距离的两个点, 考虑其中的一点关于另外一点所在簇的归属度, 若归属度满足一定阈值, 则合并两个点所在的簇。这样, 对数据集中任意两个点进行考虑, 不断将小簇进行合并, 最终对数据集形成一个划分。同时识别出异常点。通过在不同特点的人工数据集以及多个基准数据集上和多个经典距离算法进行对比实验, 实验结果说明 CMSPC 在发现任意形状簇方面是有效的。

(2) CFDPm 是对于新算法 CFDP 算法的改进。CFDP 算法不需要任何参数, 在较低的时间复杂度下能进行较高质量的聚类。然而, 当 CFDP 算法遇到多峰簇数据集上, 由于簇中心点选取不准确导致聚类算法降低。在对 CFDP 算法在多峰簇数据集聚类结果进行分析之后, 我们通过对满足一定簇间距离和簇内密度的簇, 再结合聚类内部评价指标 DBI, 合并某些簇。这样做之后使得被 CFDP 算法分开的多峰簇能被合并起来。我们在多个人工合成数据集以及基准数据集上对 CFDPm 算法进行了实验, 实验结果说明 CFDPm 算法能在多峰簇上取得较 CFDP 算法更好的聚类结果, 并具有较好的稳定性。

5.2 未来工作展望

本文的研究主要聚焦于聚类算法的研究, 提出了两个聚类算法 CMSPC 和 CFDPm。通过在多个数据集上进行实验验证了 CMSPC 和 CFDPm 聚类算法的有效性和聚类效果。然而, 本文还有多方面需要进一步研究和探索。

(1) CMSPC 聚类算法提出了一种新思路, 基于点关于簇的归属度来进行聚

类。然而该算法需要两个输入参数截断距离 T 和 M 参数。尽管给出了截断距离参数的建议范围，但是要找到最优的参数还是需要一定的经验。未来的研究中我们将考虑如何根据数据集的统计信息来更准确地确定截断距离 T 。

(2) CFDPm 算法在多峰簇数据集中，能表现出比 CFDP 算法更好的聚类结果。但是对于单峰簇数据集，CFDPm 算法并不能提高聚类质量。对于高维数据，如何对数据集的分布进行观察，进而确定数据集是单峰簇数据集还是多峰簇数据集是很有用的。现有的高维数据可视化方法并不能满足这样的要求。未来的研究会考虑对高维数据进行可视化，这不仅对 CFDPm 算法是有用的，对于聚类研究来说，能直观对高维数据进行可视化对聚类算法的选取、参数的设置都是很有用的。

(3) CFDPm 算法中的超参数 dc 尽管不那么敏感，但是对于聚类结果还是有一定的影响。未来的研究中将进一步研究超参数 dc 对于聚类结果的影响，并研究在算法中自适应调整参数 dc 。

参考文献

- [1] 韩家炜. 数据挖掘: 概念与技术[M]. 机械工业出版社, 2007: 1-19.
- [2] Bewley, Alex, and Ben Upcroft. "Advantages of Exploiting Projection Structure for Segmenting Dense 3D Point Clouds." Australian Conference on Robotics and Automation. 2013.
- [3] Uçar A, Demir Y, Güzeliş C. A new facial expression recognition based on curvelet transform and online sequential extreme learning machine initialized with spherical clustering[J]. Neural Computing and Applications, 2016, 27(1): 131-142.
- [4] Bewley, Alex, and Ben Upcroft. "Advantages of Exploiting Projection Structure for Segmenting Dense 3D Point Clouds." Australian Conference on Robotics and Automation. 2013.
- [5] Cai W, Chen S, Zhang D. Fast and robust fuzzy c-means clustering algorithms incorporating local information for image segmentation[J]. Pattern Recognition, 2007, 40(3): 825-838.
- [6] Höppner F. Fuzzy cluster analysis: methods for classification, data analysis and image recognition[M]. John Wiley & Sons, 1999.
- [7] Ester M, Kriegel H P, Sander J, et al. A density-based algorithm for discovering clusters in large spatial databases with noise[C]//Kdd. 1996, 96(34): 226-231.
- [8] Raymond T. Ng and Jiawei Han. 1994. Efficient and Effective Clustering Methods for Spatial Data Mining. In Proceedings of the 20th International Conference on Very Large Data Bases (VLDB '94), Jorge B. Bocca, Matthias Jarke, and Carlo Zaniolo (Eds.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 144-155..
- [9] Bernotas M, Karkliū K, Laurutis R, et al. The peculiarities of the text document representation, using ontology and tagging-based clustering technique[J]. Information Technology And Control, 2015, 36(2).
- [10] Larsen B, Aone C. Fast and effective text mining using linear-time document clustering[C]//Proceedings of the fifth ACM SIGKDD international conference on Knowledge discovery and data mining. ACM, 1999: 16-22.
- [11] Dhillon I S. Co-clustering documents and words using bipartite spectral graph partitioning[C]//Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining. ACM, 2001: 269-274.
- [12] Frakes W B, Baeza-Yates R. Information retrieval: data structures and algorithms[J]. 1992.
- [13] Punj G, Stewart D W. Cluster analysis in marketing research: Review and suggestions for application[J]. Journal of marketing research, 1983: 134-148.
- [14] Prayag G, Disegna M, Cohen S A, et al. Segmenting Markets by Bagged Clustering Young Chinese Travelers to Western Europe[J]. Journal of Travel Research, 2015, 54(2): 234-250.
- [15] Jiawei Han. Coursera MOOC course: Cluster Analysis in Data Mining[EB/OL]. (2015-12-17) [2016-01-05]. <https://www.coursera.org/learn/cluster-analysis>.
- [16] Jacquez G M. Spatial cluster analysis[J]. The handbook of geographic information science, 2008, 395: 416.

- [17] Varghese B M, Unnikrishnan A. SPATIAL CLUSTERING ALGORITHMS-AN OVERVIEW[J]. ASIAN JOURNAL OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY, 2013, 3(1).
- [18] MacQueen, J. Some methods for classification and analysis of multivariate observations. Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics, 281--297, University of California Press, Berkeley, Calif., 1967.
- [19] L. Kaufman and P. Rousseeuw (1987). Clustering by Means of Medoids, Statistical Data Analysis Based on the L1 Norm and Related Methods, North-Holland, 1987, pp. 405-416.
- [20] Dempster A P, Laird N M, Rubin D B. Maximum likelihood from incomplete data via the EM algorithm[J]. Journal of the royal statistical society. Series B (methodological), 1977: 1-38.
- [21] Kaufman L, Rousseeuw P J. Finding groups in data: an introduction to cluster analysis[J]. 2009.
- [22] Ng R T, Han J. CLARANS: A method for clustering objects for spatial data mining[J]. Knowledge and Data Engineering, IEEE Transactions on, 2002, 14(5): 1003-1016.
- [23] Zhang T, Ramakrishnan R, Livny M. BIRCH: an efficient data clustering method for very large databases[C]//ACM Sigmod Record. ACM, 1996, 25(2): 103-114.
- [24] Guha S, Rastogi R, Shim K. CURE: an efficient clustering algorithm for large databases[C]//ACM SIGMOD Record. ACM, 1998, 27(2): 73-84.
- [25] Karypis G, Han E H, Kumar V. Chameleon: Hierarchical clustering using dynamic modeling[J]. Computer, 1999, 32(8): 68-75.
- [26] Ankerst M, Breunig M M, Kriegel H P, et al. OPTICS: ordering points to identify the clustering structure[C]//ACM Sigmod Record. ACM, 1999, 28(2): 49-60.
- [27] Hinneburg A, Keim D A. An efficient approach to clustering in large multimedia databases with noise[C]//KDD. 1998, 98: 58-65.
- [28] Wang W, Yang J, Muntz R. STING: A statistical information grid approach to spatial data mining[C]//VLDB. 1997, 97: 186-195.
- [29] Agrawal R, Gehrke J, Gunopulos D, et al. Automatic subspace clustering of high dimensional data for data mining applications[M]. ACM, 1998.
- [30] Tran T N, Wehrens R, Buydens L M C. KNN-kernel density-based clustering for high-dimensional multivariate data[J]. Computational Statistics & Data Analysis, 2006, 51(2): 513-525.
- [31] von Borries G, Wang H. Partition clustering of high dimensional low sample size data based on p-values[J]. Computational Statistics & Data Analysis, 2009, 53(12): 3987-3998.
- [32] Rodriguez, Alex, and Alessandro Laio. "Clustering by fast search and find of density peaks." Science 344.6191 (2014): 1492-1496.
- [33] Chaoji, V.; Li, G.; Yildirim, H. & Zaki, M. J. (2011), ABACUS: Mining Arbitrary Shaped Clusters from Large Datasets based on Backbone Identification., in 'SDM' , SIAM / Omnipress, , pp. 295-306 .
- [34] Liu, Cong, et al. "Practical path-based methods for clustering arbitrary shaped data sets." Natural Computation (ICNC), 2013 Ninth International Conference on. IEEE, 2013.
- [35] Galluccio, Laurent, et al. "Clustering with a new distance measure based on a dual-rooted tree." Information Sciences 251 (2013): 96-113.

- [36] Everitt B S. Finite mixture distributions[M]. John Wiley & Sons, Ltd, 1981.
- [37] Backer E. Computer-assisted reasoning in cluster analysis[M]. Prentice Hall International (UK) Ltd., 1995.
- [38] Jain A K, Murty M N, Flynn P J. Data clustering: a review[J]. ACM computing surveys (CSUR), 1999, 31(3): 264-323.
- [39] Arthur, David, and Sergei Vassilvitskii. "k-means++: The advantages of careful seeding." Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms. Society for Industrial and Applied Mathematics, 2007.
- [40] Jain A, Dubes R. Algorithms for Clustering Data. Pentice-Hall advanced reference series[J]. 1988.
- [41] Hubert L, Arabie P. Comparing partitions[J]. Journal of classification, 1985, 2(1): 193-218.
- [42] Davies, David L., and Donald W. Bouldin. "A cluster separation measure." Pattern Analysis and Machine Intelligence, IEEE Transactions on 2 (1979): 224-227.
- [43] Bentley J L. Multidimensional binary search trees used for associative searching[J]. Communications of the ACM, 1975, 18(9): 509-517.
- [44] Pedregosa, Fabian, et al. "Scikit-learn: Machine learning in Python." The Journal of Machine Learning Research 12 (2011): 2825-2830.
- [45] Asuncion A, Newman D. UCI machine learning repository[J]. 2007.
- [46] Clustering datasets[EB/OL]. University of Eastern Finland, (2015-11-16) [2016-01-05]. <http://cs.joensuu.fi/sipu/datasets/>.
- [47] Hennig C. Package 'fpc'[J]. 2015.
- [48] Gionis, A., H. Mannila, and P. Tsaparas, Clustering aggregation. ACM Transactions on Knowledge Discovery from Data (TKDD), 2007. 1(1): p. 1-30.
- [49] Chang, H. and D.Y. Yeung, Robust path-based spectral clustering. Pattern Recognition, 2008. 41(1): p. 191-203.
- [50] Jain, A. and M. Law, Data clustering: A user's dilemma. Lecture Notes in Computer Science, 2005. 3776: p. 1-10.
- [51] G. Karypis, E.H. Han, V. Kumar, CHAMELEON: A hierarchical clustering algorithm using dynamic modeling, IEEE Trans. on Computers, 32 (8), 68-75, 1999.
- [52] Fu, L. and E. Medico, FLAME, a novel fuzzy clustering method for the analysis of DNA microarray data. BMC bioinformatics, 2007. 8(1): p. 3.
- [53] Leisch F, Leisch M F, No Z D. The mlbench package[J]. 2007.

在学期间的研究成果

一、发表论文

1. **B Wang**, YK Yao, XP Wang, XY Chen. "PB-SVM Ensemble: A SVM Ensemble Algorithm Based on SVM." Applied Mechanics and Materials. Vol. 701. Trans Tech Publications, 2014.
2. YK Yao, MW Leng, L Zhang, **B Wang**, et al. "PEnSVM: an improved SVM ensemble algorithm Based on different PCA thresholds." Journal of Information and Computational Science, v 12, n 9, p 3619-3627, June 10, 2015.
3. Ji, Dongsheng, RY Yang, L Zhang, **B Wang**, XY Chen. "The research of medical microscopic image mosaic based on the algorithm of surf." Wavelet Active Media Technology and Information Processing (ICCWAMTIP), 2013 10th International Computer Conference on. IEEE, 2013.

二、参与课题

1. 中央高校基金优秀研究生创新项目: 非均衡数据下主动学习支持向量机及其关键技术研究。(项目编号: lzujbky-2013-229)
2. 面上项目: 基于角色相似度的蛋白质相互作用网络比对关键技术研究。(项目编号: lzujbky-2014-47)

致 谢

在即将毕业的这个时候，蓦然回首，三年求学时光匆匆间就这样过去了。想当初进入兰州大学时的情景历历在目，恍如昨日，不免让人感慨光阴易逝，韶华难追。值得庆幸的是，在这3年的求学生涯中，我学到了许多让我受益的东西，无论是计算机专业相关的知识，还是做人的道理。在论文即将完成之际，想到在论文完成过程中，给了我指导和帮助的老师、同学和朋友。在此谨向他们表示衷心的感谢。

首先我要感谢我的导师陈晓云教授，三年前是您接受了我的申请，让我成为您的学生。在3年的研究生生涯中，陈教授传道、授业、解惑，给了我很大的帮助。在学术上，她要求我们遵守学术道德规范，脚踏实地做研究。在数据挖掘课程中，她教给我数据挖掘的基础知识。在本论文的实验阶段，她引导我思考，为我解惑，给了我很大启发。在撰写论文过程中，她悉心指导我，给了我很多中肯的修改意见。在此，衷心感谢陈晓云教授给与我的帮助。

感谢陈梅老师在毕业论文研究和论文撰写、修改过程中给与的指导和鼓励。是她给予的指导和建议让我的研究能够更加顺利得进行。感谢在研究生三年中给予了我帮助和关怀的博士生和硕士生师兄、师姐、师弟、师妹，感谢你们的帮助和包容，是你们给我营造了一个良好的科研氛围。在平时的实验室日常中，你们每个人都是我学习的榜样，不论是专业知识、技术，还是做人方面。

感谢和我同宿舍的三位舍友，你们生活上的帮助和关怀让我度过了这三年难忘的时光。感谢你们的帮助和包容。

感谢在这三年研究生生涯中，给予了我帮助的同学朋友。篇幅有限，在这里不一一列出。

感谢我的家人，我的父母无论在金钱还是精神上对我完成研究生生涯的支持。没有你们的无私付出，我难以取得今日的成果。